ПРОФИЛИРАНА МАТЕМАТИЧЕСКА ГИМНАЗИЯ

„КОНСТАНТИН ВЕЛИЧКОВ“ ГР.ПАЗАРДЖИК

Д И П Л О М Н А Р А Б О Т А

Тема: Разработка на уеб приложение „TravelMap“

Дипломен проект за придобиване на трета степен на професионална квалификация – част от теорията на професията

Дипломант: Боян Георгиев Байгънов

Специалност: „Приложно програмиране“ код 4810301

Професия: „Приложен програмист“ код 481030

Консултант: Ваня Шишиньова

Пазарджик

2026

Съдържание:

1. Увод 3
2. Основна част 4
2.1. Цел и задачи на дипломната работа 4
2.2. Използвани технологии 6
2.3. Реализация на проекта 7
2.3.1. Проектиране на системната архитектура и модела на данните 9
2.3.2. Модели и конфигуриране на връзки 11
2.3.3. Зависимости и правила за цялост на данните 16
2.3.4. Удостоверяване и упълномощаване 17
2.3.5. Бизнес логика 19
2.3.6. Графичен потребителски интерфейс (GUI) 26
2.3.7. Тестване и оптимизиране 35
3. Заключение 37
4. Използвана литература 38
5. Приложения 39
Ръководство за потребителя 45

1. Увод

СЛАЙД 1: Пътуванията и откриването на нови места са неразделна част от стремежа на съвременния човек към самоусъвършенстване, релаксация и културно обогатяване. Дестинациите, които посещаваме, не са просто географски точки, а източник на емоции, спомени и вдъхновение. В свят, в който информацията за забележителности, събития и маршрути е необятна, необходимостта от систематизиране и персонализиране на тази информация става все по-осезаема.

Чрез правилно подбрана дестинация и добре планиран маршрут, човек може не само да опознае света, но и да съхрани преживяванията си по начин, който позволява тяхното лесно споделяне и преразглеждане. В настоящата дигитална ера съществува огромно разнообразие от локации, категории (културни, природни, исторически) и събития, което често затруднява потребителите при избора и ефективното организиране на техните лични пътеписи и планове.

Именно поради тази причина възникна идеята за разработване на уеб приложението TravelMap, посветено на управлението и организирането на туристически дестинации. СЛАЙД 2: Въпреки че съществуват множество онлайн карти и комерсиални сайтове за резервации, повечето от тях са насочени основно към търговска дейност или обща навигация. Липсва интегрирана платформа, която да позволява на крайния потребител да създава собствена „карта на спомените“, да групира дестинации в лични каталози и да следи свързани с тях събития и статии в единна, персонализирана екосистема

2. Основна част

2.1. Цел и задачи на дипломната работа

СЛАЙД 3: Основната цел на настоящия проект е проектирането и реализацията на функционална, високопроизводителна и визуално издържана уеб платформа за управление на туристически дестинации. Системата се стреми да предостави интуитивен интерфейс за крайния потребител, като едновременно с това осигури стабилна административна рамка за управление на приложението.

За постигането на тази цел са дефинирани следните основни задачи:

1. Проектиране на мащабируем модел на данни и ORM архитектура

Първостепенна задача е изграждането на релационен модел на база данни (вж. Приложение 1: Схема на базата данни), който да позволява съхранението на сложни връзки (напр. връзки „много към много“ между дестинации и тагове). Използването на Entity Framework Core (EF Core) като ORM (Object-Relational Mapper) рамка гарантира, че достъпът до данни е абстрахиран и сигурен, предотвратявайки традиционни уязвимости като SQL инжекции. Това позволява на системата да бъде мащабируема – лесно добавяне на нови обекти (като рецензии или метеорологични данни) без нарушаване на съществуващата структура.

2. Внедряване на ASP.NET Core MVC и логическо разделение на отговорностите

Реализацията на проекта се базира на архитектурния модел MVC (Model-View-Controller). Задачата е да се постигне пълно отделяне на бизнес логиката (модели) от потребителския интерфейс (изгледи) и обработката на заявките (контролери). Това разделение улеснява екипната разработка и поддръжката, като позволява промени по дизайна да се извършват независимо от функционалната част на приложението. (вж. Приложение 2: Архитектура на приложението)

3. Система за сигурност и управление на идентичността

Ключова задача е интеграцията на ASP.NET Core Identity за управление на аутентикацията и авторизацията. Проектът предвижда внедряване на ролева система, която разграничава:

- **Администратори:** Пълен контрол над дестинациите, категориите и потребителските профили.
- **Регистрирани потребители:** Възможност за създаване на собствени каталози и добавяне на статии.
- **Гости:** Ограничен достъп само до четене на информацията.

4. Гъвкаво управление на съдържанието и динамични маршрути

Специфична задача на TravelMap е реализацията на механизъм за създаване на персонализирани каталози (маршрути). Това включва не само групиране на дестинации, но и възможност за логическо пренареждане на обектите в тях, добавяне на нови локации към съществуващи списъци и динамична визуализация на промените.

5. Разработка на административен панел (Back-office)

Системата трябва да притежава специализиран интерфейс за администратори, който да позволява бърза модерация. Задачите тук включват изграждане на CRUD (Create, Read, Update, Delete) функционалности за всички основни номенклатури: държави, региони, категории, тагове и мултимедийно съдържание (снимки).

6. Оптимизация на UI/UX и адаптивен дизайн

Една от най-важните задачи е подобряването на потребителското изживяване (UX). Това се постига чрез:

- **Интерактивност:** Използване на JavaScript за динамично превю на снимки и управление на тагове без презареждане на страницата.
- **Търсене и филтрация:** Внедряване на сложни алгоритми за филтриране по държава, категория или популярност.
- **Адаптивност:** Осигуряване на безупречна работа на приложението върху мобилни устройства и таблети чрез Bootstrap.

2.2. Използвани технологии

СЛАЙД 4: Изборът на технологии за реализацията на TravelMap е продиктуван от нуждата от сигурност, бързодействие и лесна поддръжка. В основата на проекта стои съвременната екосистема на Microsoft, съчетана с утвърдени уеб стандарти.

ASP.NET Core (Работна рамка)

ASP.NET Core е използвана ~~като високопроизводителна рамка~~ за изграждане на логиката на сървърната част. Тя осигурява вградена поддръжка за инжектиране на зависимости и е основа за внедряването на MVC модела, който разделя потребителския интерфейс от бизнес логиката.

Entity Framework Core (ORM)

EF Core ~~е избрана като обектно-релационен механизъм за картографиране (ORM). В проекта тя е използвана за абстрахиране на работата с базата данни, позволявайки на разработчика да~~ манипулира данните чрез C# обекти вместо директни SQL заявки. Чрез подхода *Code First* е генерирана цялата схема на базата данни, като EF Core управлява миграциите и сложните релации между дестинации, снимки и тагове.

Microsoft SQL Server (База данни)

За съхранение на информацията е използвана релационната система MS SQL Server. Тя осигурява надеждност ~~на транзакциите~~ и висока скорост при изпълнение на сложни заявки, необходими за филтриране на дестинациите по множество критерии (държава, категория, популярност). В базата данни се съхраняват всички текстови данни, потребителски профили и метаданни за мултимедийното съдържание.

HTML5 и CSS3 (Front-end)

Тези технологии са използвани за изграждане на скелета и визуалното оформление на страниците. CSS е разширен с персонализирани стилове за постигане на специфични ефекти, като сенки на картичките или плавни преходи при посочване, които придават модерен и завършен вид на платформата.

Bootstrap 5 (UI Framework)

Bootstrap е интегриран за постигане на пълна адаптивност. Благодарение на неговата мрежова систем, интерфейсът на TravelMap се напасва автоматично към екрани на мобилни телефони, таблети и десктоп компютри. Използвани са и компоненти като *Carousel* за галериите, *Modals* за потвърждения и *Badges* за визуална класификация на таговете.

JavaScript (Клиентска логика)

JavaScript е използван за подобряване на интерактивността. Основно приложение намира при визуализацията на качените снимки в реално време и управлението на формата за редактиране. Чрез JavaScript се реализират асинхронни заявки към сървъра – например при изтриване на снимки или обновяване на тагове, което позволява промените да се отразят веднага, без да се презарежда цялата страница.

ASP.NET Core Identity (Сигурност)

Тази специализирана библиотека е използвана за управление на сигурността. Тя покрива процесите по регистрация, вход, управление на пароли и ролева оторизация. В проекта тя е конфигурирана да поддържа разширени профили на потребителите, включващи име, фамилия и телефонен номер.

Git (Система за контрол на версията)

Версионният контрол е осъществен чрез Git, което позволява проследяване на историята на промените в програмния код и осигурява безопасно експериментиране с нови функционалности. Използването на клонове подпомага организирането на работния процес и минимизира риска от загуба на данни при системни грешки.

2.3. Реализация на проекта

СЛАЙД 5: При реализирането на уеб приложението за управление на туристически дестинации и персонализирани маршрути TravelMap бе извършено детайлно предварително проучване на съществуващи български и чуждестранни платформи, свързани с туризъм, онлайн пътеводители и интерактивни карти. Основната цел на този анализ бе да се идентифицират и извлекат добрите практики при представянето на географска информация,

методите за категоризация на забележителности, както и съвременните подходи за осигуряване на интуитивна навигация и ангажиращо потребителско изживяване.

В хода на анализа бе обърнато внимание на йерархичната структура на началните страници, визуалната идентичност на водещите портали в сектора и леснотата, с която потребителят открива специфична информация. Особен акцент бе поставен върху визуалното представяне на основните характеристики на дестинациите – географски регион, държава, категория на обекта, свързани събития и авторски статии. Проучени бяха и механизмите за филтриране и сортиране по различни критерии (популярност, дата на добавяне, тип дестинация), което допринесе за изграждането на концепция, съобразена с високите очаквания на съвременния пътешественик.

В резултат от проучването бяха дефинирани следните основни критерии за оценка и реализация на уеб приложението TravelMap:

- **Естетика и визуално въздействие:** Акцент върху висококачественото мултимедийно съдържание и изчистена начална страница, която да вдъхновява потребителя към откриване на нови места още при първото посещение.
- **Ясно дефинирана целева аудитория:** Платформата е проектирана за широк кръг от потребители – от активни туристи и блогъри, търсещи систематизация на своите преживявания, до администратори, отговорни за поддържането на актуална база данни.
- **Интуитивност на интерфейса:** Проектиране на последователен потребителски интерфейс (UI), който минимизира усилията при навигация между модулите за дестинации, събития и лични каталози.
- **Информационна пълнота и структура:** Осигуряване на добре структурирана информация, включваща не само описания, но и динамични връзки между дестинациите и контекстуално съдържание като предстоящи събития и статии.
- **Функционален достъп и сигурност:** Логическо разпределение на достъпа до основните функционалности (създаване на маршрути, редакция на данни), гарантирано чрез надеждна система за оторизация.

Настоящата разработка представя цялостния процес по проектиране и внедряване на TravelMap, описвайки използваните технологии, архитектурни решения и програмни методи. Основната идея е създаването на надеждна, мащабируема и визуално издържана платформа, която да се превърне в незаменим инструмент за управление на дигиталната "карта на спомените" на всеки потребител.

2.3.1. Проектиране на системната архитектура и модела на данните

Процесът по реализация на TravelMap започва с детайлно планиране на системната архитектура и структурата на информацията. Този етап е критичен за осигуряване на стабилност и бъдеща мащабируемост на приложението.

Дефиниране на цели и функционални задачи

Преди техническата реализация бяха ясно дефинирани приоритетите на системата. Основната задача бе да се създаде среда, в която потребителят не просто разглежда снимки, а взаимодейства с подредена база от знания. За целта бе заложено на:

- Централизирано съхранение на дестинациите.
- Мултимедийна обвързаност (галерии към всяко място).
- Контекстуална свързаност (събития и статии, реферираци към конкретна локация).

Избор на архитектурен модел (MVC)

СЛАЙД 4 + 6: За изграждането на приложението бе избран моделът Model-View-Controller (MVC). Този избор не е случаен – той позволява на разработчика да раздели проекта на три независими, но взаимодействащи си слоя:

- **Model (Модел):** Представлява данните и бизнес правилата (напр. структурата на една дестинация).
- **View (Изглед):** Потребителският интерфейс, през който хората виждат информацията.
- **Controller (Контролер):** Логическото звено, което приема заявките от потребителя, обработва ги чрез моделите и връща съответния изглед. Това разделение прави кода

по-лесен за тестване и позволява промени в дизайна (UI), без да се засяга логиката на базата данни.

Технологичен подход при базата данни (ORM и Code First)

СЛАЙД 4 + 6: Вместо традиционното ръчно писане на SQL заявки, в проекта бе внедрена ORM (Object-Relational Mapping) технологията чрез Entity Framework Core. Използван бе подходът Code First, при който структурата на базата данни се диктува от C# класовете (моделите).

- **Предимство:** Всяка промяна в моделите се отразява в базата данни чрез автоматизирани миграции, което минимизира риска от несъответствие между програмния код и данните.
- **СУБД:** Като система за управление на базата данни бе избран Microsoft SQL Server, заради неговата надеждност и отлична интеграция с .NET екосистемата.

Проектиране и нормализация на базата данни

СЛАЙД 6: Дизайнът на базата данни е съобразен с правилата за Трета нормална форма (3NF). Целта е да се избегне излишно дублиране на информация и да се гарантира целостта на данните.

- **СЛАЙД 7: Връзки (Relationships):** В приложението са реализирани сложни релации:
    - *Едно към много (1:N):* Една дестинация може да има много снимки; една държава може да съдържа много дестинации.
    - *Много към много (M:N):* Дестинациите и таговете са свързани така, че един обект може да има много тагове, а един таг може да се съдържа в много обекти.
- **Валидация на данните:** Още на етап проектиране са заложени правила за валидация. Те гарантират, че в системата не могат да влязат непълни данни (напр. дестинация без име или местоположение) и че текстовите полета имат разумни ограничения в дължината.

Реализация на CRUD функционалности

В основата на взаимодействието с потребителя стоят операциите CRUD (Create, Read, Update, Delete). Те са проектирани така, че да осигуряват плавен работен процес: от

добавянето на ново вълнуващо място, през неговото разглеждане и редактиране, до окончателното му премахване при необходимост.

2.3.2. Модели и конфигуриране на връзки

СЛАЙД 7: Моделите описват основните обекти, с които ще се изгради базата данни – потребители, дестинации, маршрути и други. Чрез използването на атрибути за валидация се гарантира, че данните ~~са консистентни и~~ защитени от некоректно въвеждане още на ниво модел.

ПРОЧЕТИ СИ МОДЕЛИТЕ, КАКВО ИМАШ, КАКВО ПРАВЯТ И ВАЛИДАЦИЯТА!!!

2.3.2.1 Модел ApplicationUser (Потребител)

Този модел представлява разширение на системния клас IdentityUser и служи за управление на профилите в приложението.

- **Функционалност:** Интегрира личните данни на потребителите със системата за сигурност и проследява тяхната активност чрез връзки с авторско съдържание.
- **Свойства и валидации:**
    1. **FirstName (Първо име):** Типът на данните е string (в базата данни NVARCHAR(100)). Полето е задължително ([Required]), с ограничение до 100 символа. Използва се за персонализация на интерфейса.
    2. **LastName (Фамилия):** Типът е string (NVARCHAR(100)). Задължително свойство с лимит от 100 символа, осигуряващо коректното изписване на пълното име на автора.
    3. **Catalogs (Списък с каталози):** Типът е List<Catalog>. Колекция, дефинираща релацията „Едно към много“. Позволява на потребителя да притежава множество лични колекции.
    4. **Destinations (Списък с дестинации):** Типът е List<Destination>. Навигационно свойство, свързващо потребителя с туристическите обекти, които е публикувал.

- **Ключове:** Наследява уникален идентификатор (GUID) от тип string, действащ като първичен ключ.

2.3.2.2 Модел Category (Категория)

Моделът дефинира класификационната структура на дестинациите в платформата.

- **Функционалност:** Групира обектите по тип (напр. „Културен туризъм“, „СПА и уелнес“), което оптимизира процеса на търсене.
- **Свойства и валидации:**
  1. **Id (Идентификатор):** Типът е int (в базата данни INT PRIMARY KEY). Използва се атрибутът [Key] за автоматично генериране на уникална стойност.
  2. **Name (Име):** Типът е string (NVARCHAR(100)). Задължително свойство с лимит от 100 символа, съдържащо името на категорията.
  3. **Destinations (Списък с дестинации):** Типът е List<Destination>. Колекция за реализиране на връзката „Едно към много“ с обектите в системата.

2.3.2.3 Модел Country (Държава)

Управлява географската принадлежност на информацията в TravelMap.

- **Функционалност:** Служи за мащабиране на приложението чрез организиране на дестинациите по политико-географски признак.
- **Свойства и валидации:**
  1. **Id (Идентификатор):** Типът е int (INT). Първичен ключ, използван за релации с външни таблици.
  2. **Name (Име):** Типът е string (NVARCHAR(100)). Задължително поле за наименованието на държавата, ограничено до 100 символа.
  3. **Destinations (Списък с дестинации):** Типът е List<Destination>. Списък с всички обекти, намиращи се в съответната държава.

2.3.2.4 Модел Tag (Таг)

Осигурява гъвкава система за етикетиране и филтриране чрез ключови думи.

- **Функционалност:** Позволява на потребителите да намират места по специфични белези (напр. „подходящо за деца“, „безплатен вход“).
- **Свойства и валидации:**
    1. **Id (Идентификатор):** Типът е int (INT). Първичен ключ на етикета.
    2. **Name (Име):** Типът е string (NVARCHAR(100)). Задължително свойство (макс. 100 символа), визуализирано в интерфейса като интерактивен етикет.
    3. **Destinations (Списък с дестинации):** Типът е List<Destination>. Управлява релацията „Много към много“ с туристическите обекти.

2.3.2.5 Модел Destination (Дестинация)

Това е централният и най-мащабен модел в софтуерната архитектура на проекта.

- **Функционалност:** Обединява в себе си всички данни за конкретен туристически обект, включително локация, описание, авторство и мултимедия.
- **Свойства и валидации:**
    1. **Id (Идентификатор):** Типът е int. Първичен ключ на обекта.
    2. **Name (Име):** Типът е string (NVARCHAR(100)). Задължително поле за наименование на обекта.
    3. **CountryId / CategoryId:** Типът е int. Външни ключове, сочещи към съответните таблици за държава и категория.
    4. **Region (Регион):** Типът е string (NVARCHAR(100)). Задължително поле за уточняване на областта или общината.
    5. **CreationDate (Дата на създаване):** Типът е DateOnly (в базата данни DATE). Автоматично генерирана дата на публикуване.
    6. **Description (Описание):** Типът е string (NVARCHAR(MAX)). Позволява до 10,000 символа текст за детайлно представяне.
    7. **Popularity (Популярност):** Типът е int. Динамично свойство (изчислимо), което не се записва в БД, а преброява включванията в потребителски каталози.

8. **UploadedImages (Качване):** Типът е List<IFormFile>. Поле, маркирано с [NotMapped], използвано само по време на трансфера на файлове към сървъра.
9. **Навигационни списъци:** Включва колекции за снимки (Images), събития (Events), статии (Articles) и връзки към каталози (CatalogDestinations).

2.3.2.6 Модел DestinationImage (Изображение)

Управлява визуалните ресурси, съхранявани в файловата система на приложението.

- **Функционалност:** Свързва физическите файлове на сървъра с конкретни записи в базата данни.
- **Свойства и валидации:**
    1. **Id (Идентификатор):** Типът е int. Уникален номер на изображението.
    2. **ImageUrl (URL):** Типът е string (NVARCHAR(MAX)). Задължително поле, съхраняващо относителния път до файла.
    3. **DestinationId:** Типът е int. Външен ключ, дефиниращ собственика на изображението.

2.3.2.7 Модел Event (Събитие)

Представлява динамичния компонент на платформата, добавяйки времева стойност към дестинациите.

- **Функционалност:** Предоставя информация за предстоящи прояви на територията на обектите.
- **Свойства и валидации:**
    1. **Name (Име):** Типът е string (NVARCHAR(200)). Задължително заглавие на събитието.
    2. **Date (Дата):** Типът е DateTime (в базата данни DATETIME2). Определя кога ще се състои събитието.
    3. **DestinationId:** Типът е int. Външен ключ към таблицата на дестинациите.

2.3.2.8 Модел Article (Статия)

Служи за разширено текстово съдържание и споделяне на опит.

- **Функционалност:** Обогатява платформата с пътеписи и новини, свързани с конкретни локации.
- **Свойства и валидации:**
    1. **Title (Заглавие):** Типът е string (NVARCHAR(200)). Задължително свойство за заглавие на публикацията.
    2. **Content (Съдържание):** Типът е string (NVARCHAR(MAX)). Основният текст на статията.
    3. **DestinationId:** Типът е int. Външен ключ, сочещ към свързаната дестинация.

2.3.2.9 Модел Catalog (Каталог)

Това е основната единица за персонализация на потребителското пространство.

- **Функционалност:** Позволява на регистрираните потребители да организират дестинациите в собствени маршрути или списъци с желания.
- **Свойства и валидации:**
    1. **UserId (Потребител ИД):** Типът е string. Външен ключ към таблицата на потребителите.
    2. **Name (Име):** Типът е string (NVARCHAR(100)). Задължително наименование на колекцията.
    3. **IsPublic (Публичен):** Типът е bool (BIT). Определя дали каталогът е видим за другите потребители.
    4. **Notes (Бележки):** Типът е string. Незадължително поле за лични записки към каталога.

2.3.2.10 Модел CatalogDestination (Свързващ модел)

Технически модел, реализиращ сложната връзка между каталози и обекти.

- **Функционалност:** Позволява добавянето на метаданни към връзката „Много към много“, като подредба на дестинациите.
- **Свойства:**

1. **CatalogId / DestinationId:** Типът е int. Композитни ключове, осигуряващи релацията между двата модела.
2. **DisplayOrder (Ред):** Типът е int. Свойство, определящо последователността на дестинациите в рамките на каталога.

2.3.2.11 Модел DestinationSpecifications (Спецификации)

Този модел функционира като ViewModel и не се записва в базата данни.

- **Функционалност:** Служи като контейнер за данни при филтриране и търсене.
- **Свойства:** Съдържа колекции IEnumerable от държави, категории и тагове, които се изпращат към изгледите за запълване на навигационните и филтриращи елементи в интерфейса.

2.3.3. Зависимости и правила за цялост на данните

Тази секция описва правилата, по които системата реагира при изтриване на свързани записи. Тези конфигурации са критични за предотвратяване на грешки от тип „остатъчни данни“ (orphaned data) (вж. Приложение 3: Установяване на зависимости между таблиците).

Рестриктивно изтриване на държави и категории

Конфигурирано е правило за връзките на дестинацията с Country и Category, където системата блокира изтриването на държава или категория, ако в базата данни все още съществуват дестинации, свързани с тях. Това предпазва потребителя от случайно унищожаване на важни структурни данни и гарантира, че всяка дестинация винаги ще има валидна локация и класификация.

Управление на авторството при изтриване на потребител

При изтриване на потребителски акаунт, ако авторът на дадена дестинация бъде премахнат от системата, самата дестинация остава в базата данни, но полето AuthorId се променя на null. Така съдържанието се запазва за платформата, превръщайки се в анонимно, вместо да бъде изтрито заедно с профила.

Каскадно изтриване на мултимедия и потребителски ресурси

За моделите DestinationImage и Catalog е внедрено каскадно изтриване. При изтриване на дестинация, всички свързани с нея записи за изображения се премахват автоматично. Аналогично, ако потребител бъде изтрит, неговите лични каталози се заличават веднага, за да не заемат излишно място в сървъра.

Управление на релацията CatalogDestination

Тази свързваща таблица е конфигурирана със сложен първичен ключ, съставен от CatalogId и DestinationId. Приложено е двойно каскадно изтриване. Ако бъде изтрит каталог, се премахват всички негови връзки към дестинации. Ако бъде изтрита конкретна дестинация, тя автоматично се премахва от всички потребителски каталози, в които е била включена. Това поддържа списъците на потребителите винаги актуални.

Жизнен цикъл на статии и събития

За моделите Article и Event е настроено каскадно изтриване спрямо дестинацията. Тъй като тези обекти са логически зависими от конкретно местоположение, премахването на дестинацията от картата води до автоматично заличаване на всички статии, пътеписи и събития, свързани с нея.

2.3.4. Удостоверяване и упълномощаване

СЛАЙД 8: Процесът по осигуряване на защитен достъп е разделен на няколко нива: идентификация на самоличността, автоматизирана валидация и разпределяне на нивата на достъп чрез административни зони.

Процес на регистрация и управление на профили

За разлика от стандартните системи, регистрационната форма на TravelMap е разширена, за да събира по-богата информация за потребителите още при самото им присъединяване към платформата.

- **Събиране на данни:** Потребителят е длъжен да въведе своето собствено име (FirstName) и фамилия (LastName), които се съхраняват в разширения модел ApplicationUser. Допълнително се изискват телефонен номер и електронен адрес, които служат за контакт и идентификация.

- **Сигурност на паролите:** Системата налага политики за сигурност, които изискват минимална дължина на паролата, наличие на главни букви, цифри и специални символи. Паролите се съхраняват в базата данни в хеширан вид, което гарантира, че дори при пробив в данните, реалните пароли остават защитени.
- **Автоматично потвърждение:** С цел подобряване на потребителското изживяване и бърз достъп до функционалностите, системата е конфигурирана автоматично да потвърждава имейла и телефона на всеки нов потребител в момента на неговото създаване. Това премахва необходимостта от изчакване на верификационни съобщения.

Разделение на приложението чрез "Areas" (Зони на достъп)

За постигане на ясна архитектура и висока сигурност, проектът е разделен на три логически области. Всяка от тях има специфично предназначение и нива на оторизация:

1. **Identity (Идентичност):** Тази зона капсулира цялата логика по вход, регистрация, изход и управление на профила. Тя е изолирана, за да се гарантира, че чувствителните операции по сигурността се обработват централизирано и защитено.
2. **User (Потребителска зона):** Предназначена за регистрираните потребители на платформата. Тук те имат достъп до създаването на лични каталози, управление на своите дестинации и преглед на личната си история. Достъпът до тази зона е защитен чрез атрибута за авторизация.
3. **Admin (Административна зона):** Това е най-високото ниво на достъп в системата. В нея се намират контролните панели за управление на категории, държави, тагове и модерация на потребителското съдържание. Тази зона е достъпна само за потребители с роля "Admin".

Ролева система и упълномощаване

Упълномощаването (оторизацията) определя какво може да прави всеки потребител в системата. В TravelMap е внедрен моделът „*Базиран на роли контрол на достъпа*“ (RBAC):

- **Роля "User":** Стандартен потребител, който може да чете обща информация и да създава собствени маршрути.

- **Роля "Admin":** Има право да извършва деструктивни действия (изтриване на записи), да управлява системни номенклатури и да променя настройките на приложението.
- **Политики за сигурност:** В контролерите са внедрени атрибути, които автоматично пренасочват неоторизирани потребители към страницата за вход, ако се опитат да достъпят забранена за тях зона (например административния панел).

2.3.5. Бизнес логика

СЛАЙД 9: Бизнес слоят представлява „свързващото звено“ в архитектурата на TravelMap. Неговата основна роля е да прихване потребителските заявки, да извлече или запише необходимите данни чрез модела и да върне подходящ отговор към потребителя. В този проект контролерите са организирани чрез ~~модулната система на~~ Areas, което позволява стриктно разделение между ~~системните функции,~~ потребителските възможности и административните инструменти.

При разработката на контролерите са спазени принципите на инжектиране на зависимости, като през конструкторите са подадени необходимите услуги като TravelMapContext за достъп до базата данни и UserManager за работа с потребителските акаунти.

2.3.5.1 Модул за удостоверяване

Тази зона е фундаментална за сигурността на приложението. Тук са капсулирани всички процеси, свързани с легитимирането на потребителите.

- **Функционалност „Регистрация“ (Register):** Този метод обработва въвеждането на нови потребители в системата. Логиката включва приемане на данни от разширената форма (име, фамилия, имейл, телефон, парола), създаване на обект от тип ApplicationUser и записването му в базата данни. Тук се извършва и автоматичното потвърждаване на профила, което е специфично решение за проекта с цел по-бърз достъп до услугите.

- **Функционалност „Вход“ (Login):** Методът отговаря за автентикацията. Той проверява предоставените идентификатори спрямо съхранените данни в SQL Server. При успешна проверка се генерира сесийна бисквитка, която поддържа потребителя „вписан“ при преминаване между различните страници.
- **Функционалност „Изход“ (Logout):** Изчиства потребителската сесия и унищожава автентикационния токен, гарантирайки защитата на данните след приключване на работа с платформата.

2.3.5.2 Потребителски модул

Тази секция обхваща логиката, достъпна за крайните потребители, включително информационни страници, управление на събития, четене на статии и организиране на лични маршрути (каталози).

HomeController (Навигационен контролер)

Този контролер служи за обслужване на статичните и началните страници в потребителската зона.

- **Метод Index:** Зарежда началната точка на потребителския интерфейс.
- **Метод Tips:** Специализирана страница, предоставяща съвети за пътуване. Тя подобрява информационната стойност на платформата.
- **Управление на грешки:** Методът използва ResponseCache настройки, за да предотврати кеширането на страници с грешки, и предоставя детайлен RequestId за диагностика чрез модела ErrorViewModel.

EventsController

Контролерът отговаря за визуализацията на всички предстоящи прояви, свързани с туристическите обекти.

- **Интелигентно зареждане на данни:** В метода Index се прилага каскадно включване на свързани данни. Това позволява в списъка със събития веднага да се покаже снимка на мястото, където ще се проведе събитието, и държавата, в която се намира.

- **Хронологична подредба:** Събитията автоматично се сортират по дата, за да може потребителят първо да вижда най-скорошните прояви.

ArticlesController

Този контролер обработва дългите текстови статии и пътеписи.

- **Оптимизация на паметта:** В метода Index е внедрена специфична оптимизация чрез Select. Вместо да се теглят всички данни, системата избира само необходимите компоненти (Статия, Дестинация, Държава и само първото изображение).
- **Логика за водеща снимка:** Чрез C# логика контролерът гарантира, че в прегледа на статиите се визуализира точно едно изображение (първото по ред), което пести трафик и поддържа визуалната чистота на списъка.

DestinationsController

Тук е внедрена най-сложната логика за извличане и сортиране на данни (вж. Приложение 4: Сортиране и оптимизиране на заявки при дестинациите).

- **Динамично сортиране:** Методът Index поддържа множество критерии за подредба: по име, по дата на добавяне, по държава и по популярност. Използва се ViewData за пренос на параметрите на сортиране между контролера и изгледа, което позволява на потребителя да превключва между възходящ и низходящ ред.
- **Сложна заявка (Анонимна):** Преди финалното извличане, контролерът създава анонимен обект, който изчислява броя на снимките и броя на добавянията в каталози (PopCount). Това е "Popularity" алгоритъмът на приложението.
- **Оптимизация на SQL заявките:**
    - **AsNoTracking():** Използва се за заявки само за четене, което значително ускорява работата на Entity Framework, тъй като той не следи промените по обектите.
    - **AsSplitQuery():** Тази оптимизация разделя голямата заявка с много Include клаузи на няколко по-малки SQL заявки. Това предотвратява "картезианска

експлозия" и подобрява скоростта при работа с големи обеми от свързани данни (Tags, Images, Author).

CatalogsController

Този контролер реализира една от най-важните социални и организационни функции на приложението. Каталогът тук се разглежда като личен маршрут на потребителя.

- **Публични и лични каталози:** Системата поддържа два типа колекции. Методът Index показва само личните каталози на вписания потребител (чрез GetUserId), докато методът Public е достъпен за всички (чрез [AllowAnonymous]) и показва споделените маршрути на други пътешественици.
- **Функция ToggleDestination (Превключвател):** Тази функция позволява на потребителя с един бутон да добавя или премахва дестинация от даден каталог. Контролерът проверява дали записът съществува в CatalogDestinations: ако да – го изтрива, ако не – го създава. След операцията потребителят се пренасочва обратно към предходната страница чрез Referer заглавието на HTTP заявката.
- **Пренареждане на маршрута (Reorder):** Тъй като каталозите често служат за планиране на пътуване, редът на местата е важен. Методът ReorderDestination позволява на потребителя да мести дестинация "нагоре" или "надолу" в списъка. Логиката работи чрез размяна на свойството DisplayOrder между текущия и съседния елемент в базата данни.
- **Сигурност и Оторизация:** Всички методи за модификация са защитени с [Authorize] и проверяват дали текущият потребител е собственик на каталога, преди да позволят промени.

2.3.5.3 Административен модул

Административните контролери осигуряват пълния жизнен цикъл (CRUD операции) на данните в системата.

ApplicationUsersController (Управление на потребители)

Този контролер управлява профилите на всички регистрирани потребители в системата.

- **Създаване (Create):** Използва UserManager<ApplicationUser> за сигурно хеширане на пароли и създаване на потребител. При създаването на акаунт от панела са деактивирани изисквания за валиден телефон и имейл.
- **Редактиране (Edit):** Администраторът може да променя лични данни (имена, имейл, телефон). Контролерът актуализира обекта в базата и синхронизира UserName с новия имейл адрес за консистентност при вход.
- **Изтриване (Delete):** Администраторът може да променя деактивира потребителски акаунти, като изтрива всичките данни за тях от базата данни.

Articles и Events Controllers

Тези два контролера са огледални и отговарят за "захранването" на платформата с информация.

- **Връзка с дестинации:** При създаване на статия или събитие, администраторът избира дестинация от динамичен SelectList. Това гарантира, че всяко съдържание е логически обвързано с конкретно място на картата.
- **Details и Delete:** Предоставят административен преглед и възможност за премахване на остаряла информация с потвърждение (DeleteConfirmed).

Categories и Countries Controllers

Тези контролери управляват основните класификатори на приложението.

- **Групово управление:** Те са оптимизирани за бърза работа. След всяко действие (Create, Edit, Delete), системата пренасочва администратора към общия изглед DestinationSpecifications, където се управляват всички системни справочници на едно място.

CatalogsController (Административен контрол върху маршрути)

За разлика от потребителския контролер, тук администраторът има права над всички каталози, независимо кой ги е създал.

- **Функция RemoveDestination:** Позволява на администратора да модерира съдържанието на маршрутите, като премахва неподходящи или несъществуващи дестинации.
- **Логика за пренареждане (Reorder):** Това е ключов механизъм за подредба на маршрута. Логиката работи по следния начин:
    1. **Извличане:** Вземат се всички дестинации в каталога, подредени по текущия им DisplayOrder.
    2. **Идентификация:** Намира се индексът на дестинацията, която искаме да преместим.
    3. **Избор на цел (Target):** При посока "up" (нагоре), целта е елементът с индекс -1, а при "down" (надолу) – елементът с индекс +1.
    4. **Размяна (Swap):** Системата извършва класическа размяна на стойностите на DisplayOrder. Стойността на текущия елемент отива в целевия, а временна променлива (tempOrder) прехвърля старата стойност на целевия към текущия.
    5. **Запис:** Промените се запазват в базата, което визуално "премества" обекта в списъка.

TagsController (Управление на етикетите)

Този контролер отговаря за поддръжката на метаданните, които категоризират съдържанието.

- **Директен работен поток:** Тъй като таговете са спомагателен елемент, контролерът е настроен да пренасочва администратора директно към DestinationSpecifications след всяко действие (Create, Edit, Delete). Това позволява бързо управление на множество категории и етикети от един общ интерфейс.

DestinationsController (и управление и мултимедийната логика)

Това е централният модул, в който е внедрена комплексната логика за работа с файловата система и свързаните таблици.

- **Мултимедийна архитектура (Images):**

- **Прикачване на множество файлове:** Използва IFormFileCollection, за да позволи на администратора да качва неограничен брой снимки едновременно. Системата автоматично генерира уникални имена (Guid), за да избегне конфликти в директорията „*wwwroot/images/destinations*“.
- **Интелигентно триене:** В метода Edit е внедрена функционалност за селективно премахване. Чрез списъка deletedImageIds, контролерът идентифицира снимките, които трябва да бъдат премахнати. Логиката включва както изтриване на записа от базата данни, така и физическо премахване на файла от сървъра, поддържайки хостинг пространството чисто.
- **Пълно почистване при изтриване:** При премахване на цяла дестинация (DeleteConfirmed), контролерът итерира през всички прикачени изображения и изтрива техните файлове от диска, преди да заличи самата дестинация.

- **Управление на тагове (Many-to-Many):** Системата използва динамичен списък selectedTags. При редакция, контролерът първо изчиства старата колекция и записва новата селекция, осигурявайки актуалност на връзките между обектите и техните характеристики.

DestinationSpecificationsController (Обединение на малки таблици)

Този контролер служи като централно табло за управление на всички номенклатури (вж. Приложение 5: Групиране на характеристиките на дестинация в общ обект).

- **Агрегирано зареждане:** Методът Index конструира комплексен модел DestinationSpecifications, който обединява данни от три различни таблици (Държави, Категории и Тагове). Всички те се извличат с сортиране по име, за да се осигури лесна навигация за администратора.

Ключови технически оптимизации:

1. **AsSplitQuery():** Използва се при зареждане на дестинации, за да се предотврати т.н. "Картезианска експлозия". Вместо голяма и бавна заявка, EF Core разделя извличането на снимки, тагове и описания на няколко по-ефективни SQL заявки.

2. **Извличане само на полетата, които трябва да се обновят:** Това позволява обновяване на основните полета (име, описание, регион) без нужда от пълно презареждане на обекта, което подобрява производителността.
3. **Управление на пътищата:** Пътищата до снимките се съхраняват като относителни стрингове (/images/destinations/...), което прави приложението лесно преносимо между различни сървъри и среди.

2.3.6. Графичен потребителски интерфейс (GUI)

СЛАЙД 10: GUI представлява изгледите, от където потребителите взаимодействат със сървъра като изпращат команди (заявки) до него. Той е изграден с Razor Pages и Bootstrap 5. Дизайнът е проектиран да бъде адаптивен и динамичен, променяйки се в реално време според правата на достъп на потребителя.

2.3.6.1 Глобален Layout (Структурен скелет)

Той служи като централна точка за управление на потребителското преживяване и ресурсите на приложението, като в него са дефинирани файловете с преизползваем код за всеки изглед (Bootstrap и JavaScript). Представлява хедър и футър, които ще се показват на потребителя от всяка страница, която отваря. Използва се като навигационен панел.

- **Динамична навигация (интерфейс, базиран на роли**~~): Използва се UserManager, за да се персонализира интерфейсът.~~ Ако потребителят е вписан, системата извлича неговото име за персонално приветствие в хедъра.
    - **Условна видимост на менюта:** Логиката разделя публичното от частното съдържание – нерегистрираните потребители виждат бутони за вход/регистрация, списъка с дестинациите със статиите и събитията към тях и публичните каталози, докато вписаните получават достъп до секция „Моите каталози“.
- **Административна изолация:** Внедрен е блок, който визуализира „Административен панел“, само когато ролята на текущия потребител е администраторска. Това е критична точка

за сигурност на интерфейса – администраните линкове за управление на потребители, дестинации и характеристики са напълно скрити за обикновения потребител.

- **Сигурност при отписване:** Logout функционалността е реализирана чрез POST форма със зона "Identity". Това е стандарт за сигурност, който предотвратява злонамерени опити за отписване на потребителя чрез обикновен линк (GET заявка).

Визуална стилизация

В _Layout.cshtml е вградена специфична CSS логика за модерна визия:

- **Микро-интеракции:** Бутоните за аутентикация имат добавен hover ефект с мащабиране. Това подобрява интерактивността и насочва вниманието на потребителя към основните действия.
- **Sticky Navigation:** Използването на класа sticky-top гарантира, че навигацията винаги е достъпна, което е от ключово значение при дълги страници с дестинации.

Клиентска логика и зареждане на ресурси (Скриптове)

Скриптовият блок в края на файла е организиран за максимално бързо зареждане на страницата:

- **Библиотеки:** Зареждат се jQuery и Bootstrap Bundle, които управляват интерактивните елементи като падащи менюта и мобилното бургер-меню.
- **RenderSectionAsync:** Този механизъм позволява на конкретните страници (например страницата с дестинации или регистрация) да инжектират свои специфични скриптове (валидации, карти, галерии) точно в края на тялото, без да натоварват глобалния шаблон.

2.3.6.2 Гост-потребител

Това са изгледите, до които има достъп всеки, отворил сайта, в това число и нерегистрирания потребител (гост). Дизайнът е изграден така, че да бъде максимално интуитивен, като използва комбинация от Razor за динамично съдържание и JavaScript за светкавична скорост при филтриране без презареждане на страниците.

Начална страница и Полезни съвети (Home)

Началната страница е първото нещо, което всеки потребител вижда от уебсайта, докато секцията със съвети добавя практическа стойност.

- **Интерактивни съвети:** Вместо просто четене, тук е внедрена QuickSubmit форма. Тя позволява на всеки потребител да изпрати съвет директно към контролера на статиите, превръщайки статичната страница в инструмент за събиране на съдържание от общността.

Каталог с Дестинации (Index)

Това е най-комплексният изглед, който събира данни от няколко таблици (Images, Tags, Countries) и ги представя чрез „флаш карти“.

- **Логика за снимки:** В кода е заложен механизъм, който гарантира, че сайтът никога няма да изглежда „счупен“. Използва се проверка ако администраторът е забравил да качи снимка, системата автоматично визуализира изображение по подразбиране.
- **Интеграция с колекции:** Бутонът „Колекции“ на всяка карта работи динамично. Ако потребителят е гост, вижда покана за вход. Ако е логнат, се зареждат неговите лични каталози, а иконите се сменят в реално време (плюс или минус) в зависимост от това дали дестинацията вече е добавена там.
- **Визуални индикатори:** Използват се полупрозрачни значки, които стоят върху снимката и показват броя кадри в галерията и таговете, без да заемат място в текстовата част.

Детайлен изглед на дестинация

Това е „профилът“ на избраното място, като тук логиката се фокусира върху визуалното съдържание и метаданните.

- **Динамичен Carousel (Галерия):**
    - Внедрена е интелигентна проверка на броя снимки. Ако дестинацията има повече от едно изображение, системата автоматично активира навигационните стрелки и индикаторите в долната част.

- Изображение по поразбиране**:** Отново е заложен защитен механизъм – ако в базата данни липсват снимки, се зарежда системното default.jpg, за да се запази структурата на страницата.
- Снимките са форматирани, за да изглеждат еднакво добре в рамките на 450px височина, независимо от оригиналните им пропорции.

- **Модул „Предстоящи събития“:**
  - Системата итерира през колекцията Events. Ако има активни събития, те се визуализират като интерактивни карти с подчертана дата.
  - Всеки елемент има директна връзка към детайлите на събитието, което улеснява навигацията в портала.
- **Модул „Статии и Пътеписи“:**
  - Тук се показват текстове, написани от потребители специално за това място.
  - За по-чист интерфейс е добавена логика за отрязване на текста до първите 120 символа. Това гарантира, че прегледът на статиите ще бъде с еднаква дължина, независимо колко дълъг е оригиналният текст.

JavaScript Логика (Филтриране от страна на клиента)

За да се постигне усещане за модерно приложение, търсенето и филтрирането се случват изцяло в браузъра на потребителя, **без излишни заявки към сървъра**.

Търсене в реално време: Всяка дестинация има скрит атрибут data-search. В него са събрани името, държавата, таговете и автора. Когато потребителят пише в търсачката, скриптът просто сравнява текста с този атрибут и скрива несъвпадащите елементи чрез d-none (вж. Приложение 6: Филтриране в реално време).

- **Филтър за тагове:** Бутоните за тагове (например #планина, #море) работят чрез функцията „filterByTag“. Тя автоматично попълва търсачката и „пресява“ дестинациите само с един клик, като същевременно визуално маркира активния таг.
- **Състояние без резултати:** Скриптът следи броя на видимите карти. Ако той падне до нула, автоматично се показва съобщение за липса на резултати с опция за изчистване на филтрите.

Блог и Събития (Articles & Events)

Тези секции се фокусират върху четенето и информираността, като използват стандартни Index и Details изгледи.

- **Index (Списък):** Тук се прилага логиката за „водеща снимка“, където се излиза само първото изображение на дестинацията, за която се отнася статията/събитието, което оптимизира скоростта на зареждане на списъка.
- **Details (Пълен преглед):** Тук се визуализира пълното съдържание на статиите или детайлите за събитието (дата, локация, описание). Интегрирана е логика за показване на автора и датата на създаване, за да се придаде автентичност на съдържанието.

Публични каталози

Този интерфейс позволява на общността да разглежда чужди маршрути без необходимост от редакция.

- **Филтрирано съдържание:** Чрез заявка се визуализират само каталози, чийто атрибут IsPublic е настроен на true. Резултатите са сортирани по хронологичен ред чрез, за да се покажат най-новите идеи най-отгоре.
- **Визуална йерархия:** Всяка карта ясно показва името на създателя, броя дестинации и кратки бележки, предоставяйки контекст преди самото разглеждане.
- **Ограничен достъп:** В този изглед бутоните за редактиране, триене и пренареждане са софтуерно премахнати. Потребителите имат достъп само до бутона „Разгледай“, който води към детайлна страница в режим на четене (Read-only).

2.3.6.3 Управление на идентичността

Следващия дял е когато потребителят реши да създаде или влезе със съществуващ акаунт. Интерфейсите за автентикация и оторизация, изградени чрез ASP.NET Core Identity. Дизайнът е фокусиран върху сигурността и потребителското изживяване.

Регистрация (Register)

Изгледът за регистрация позволява създаването на нови акаунти, като разширява стандартния модел с допълнителни метаданни.

- **Разширени данни:** Формата събира не само системни данни (имейл и парола), но и персонална информация като име, фамилия и телефонен номер, които се мапват към ApplicationUser.
- **Интерактивна проверка:** В реално време, чрез секцията Scripts, се активира клиентска валидация. Тя предотвратява изпращането на невалидни заявки към сървъра, проверявайки за съвпадение на паролите и коректност на имейл формата.

Вход и сесия (Login)

Интерфейсът за вход е проектиран да бъде изчистен и функционален, осигурявайки бърз достъп до личните ресурси на потребителя.

- **„Запомни ме“ функционалност:** Внедрена е логика за управление на бисквитките, която позволява на потребителя да запази сесията си активна за определен период.
- **Тестови данни за достъп:** За целите на разработката са конфигурирани два типа профили:
    - **Администраторски акаунт:** *admin@travel.com* с парола *Admin123!*
    - **Потребителски акаунт:** *user@travel.com* с парола *User123!*

Сигурен изход и край на сесия (Logout)

Този изглед управлява безопасното прекратяване на потребителската сесия.

- **Двойно потвърждение:** Вместо директен линк, изходът се осъществява чрез POST заявка, в случай на натискане по невнимание от потребителя.
- **Контекстуална обратна връзка:** Интерфейсът динамично променя съдържанието си – ако потребителят е влязъл, изисква потвърждение, а ако вече е излязъл, визуализира съобщение за успех с опции за връщане към началната страница.

2.3.6.4 Обикновен потребител

След успешна аутентикация, потребителят получава достъп до персонализирана секция „Моите Колекции“. Тук интерфейсът се трансформира, позволявайки динамично управление на лични маршрути и планове за пътуване.

Изглед на колекциите (Index)

Този изглед визуализира всички папки с дестинации, създадени от потребителя.

- **Визуален стил „Папка“:** Използван е CSS компонент, който придава на всяка карта вид на физическа папка. Дизайнът включва и визуални индикатори за статус (Публичен/Личен) чрез цветни значки.
- **Динамични метаданни:** Всяка карта показва в реално време броя на включените дестинации и кратки бележки, като използва за поддържане на чиста мрежова структура.

Детайли и интерактивно подреждане (Details)

Това е най-комплексната част от потребителския каталог, където се извършва фината настройка на съдържанието.

- **Контрол на достъпа:** Чрез Claims системата проверява дали текущият потребител е собственик на каталога. Само при съвпадение или при администраторски права се визуализират бутоните за модификация.
- **Логика за разместване:** Внедрена е система за промяна на реда на дестинациите чрез бутони „Нагоре“ и „Надолу“, позволявайки на потребителя да планира последователността на своето пътуване.
- **Управление на съдържанието:** Всяка дестинация в списъка разполага с бутон за бързо премахване, който действа моментално върху релацията в междинната таблица.

Създаване, редактиране и изтриване (Edit, Create, Delete)

Формите за създаване, редактиране и изтриване са фокусирани върху три основни аспекта на каталога:

- **Име и идентификация:** Валидирано текстово поле за заглавие на колекцията.

- **Публичност:** Чрез кутия за маркиране потребителят решава дали каталогът да бъде личен или да бъде споделен с останалата част от общността в публичната секция.
- **Бележки и планове:** Разширено текстово поле за добавяне на детайлни описания или логистични бележки към конкретния списък.
- **Сигурност при изтриване:** Операцията за премахване на цял каталог е защитена с потвърдителен изглед, предотвратяващ случайно изтриване на данни.

2.3.6.5 Административен панел и управление на съдържанието

Административният панел е „сърцето“ на управлението на TravelMap. Той предоставя пълен контрол върху номенклатурите на системата и потребителското съдържание.

Глобални номенклатури и потребители

За основните обекти в системата — Статии (Articles), Събития (Events), Тагове (Tags), Държави (Countries) и Категории (Categories) — е реализиран стандартен CRUD (Create, Read, Update, Delete) интерфейс.

- **Управление на потребители:** Администраторът има достъп до списък с всички регистрирани потребители (ApplicationUser), като може да разглежда техните детайли, да редактира профилите им или да ограничава достъпа им при необходимост.
- **DestinationSpecifications:** Това е специализиран изглед за дефиниране на техническите характеристики и филтри, които по-късно се асоциират с дестинациите.

Управление на дестинации (Destinations)

Управлението на дестинациите е най-комплексната част от административния панел.

- **Index и Details:** Списъчен изглед с разширени филтри и бърз преглед на основните параметри.
- **Delete:** Сигурно изтриване с каскадно премахване на свързаните изображения от сървъра.
- **Създаване и Редактиране (Фокус върху изображенията)**

Модулите за Create и Edit на дестинации са проектирани да обработват голям обем от метаданни и мултимедия едновременно. Системата използва комбиниран подход от C# (Backend) и JavaScript (Frontend) за управление на файлове:

- **Динамично зареждане:** Използва се FileReader API, което позволява на администратора да види преглед (preview) на избраните снимки още преди те да бъдат изпратени към сървъра.
- **„Влачи и пусни“ зона:** Интегрирана е интерактивна зона, която поддържа влачене на множество файлове едновременно.
- **Интелигентно синхронизиране:** Чрез обекта DataTransfer в JavaScript, списъкът с избрани файлове се актуализира в реално време, позволявайки премахване на конкретна снимка от опашката за качване, преди финалното записване.

При режим на редакция (Edit), интерфейсът разделя логически текущите снимки от новите:

- **Маркиране за изтриване:** Вместо моментално изтриване, администраторът маркира съществуващите снимки. Визуално те стават сиви, а скрито поле предава техните ID-та към контролера. Това предотвратява случайни загуби на данни.
- **Добавяне на нови:** Новите изображения се качват паралелно, без да засягат старата структура, докато формата не бъде изпратена.

Модерация на каталози (Catalogs)

За разлика от обикновения потребител, администраторът разполага с глобален поглед върху всички каталози в системата:

- **Пълен достъп:** Администраторът може да редактира или трие всеки каталог, независимо дали е личен или публичен.
- **Разширени данни:** В списъчния изглед на администратора се визуализират данни за собственика (Email/User), статуса на видимост и системни бележки, които не са достъпни за крайните потребители.
- **Табличен изглед:** Използва се оптимизирана таблица с btn-group за бързи действия, осигуряваща лесна навигация при голям обем от записи.

2.3.7. Тестване и оптимизиране

СЛАЙД 11: По време на разработката на всеки модул се прилагаше паралелно тестване за логика, валидация на данните, адаптивност на интерфейса и цялостното UI/UX преживяване. Този итеративен подход позволи грешките да бъдат отстранявани в зародиш, вместо да се натрупват в крайния етап.

Проверки на логиката и сигурността

След реализирането на връзките между потребители, каталози и дестинации, фокусът беше върху интегритета на информацията:

- **Авторизация:** Проверени са правата за достъп, за да се гарантира, че личните данни са видими само за техния собственик, а административните функции са защитени зад ролеви филтри.
- **IDOR защита:** Беше отстранен рискът от неоторизиран достъп през URL адреса чрез въвеждане на допълнителни сървърни проверки с контрол на достъпа до страниците.

Валидация и работа с ресурси

Особено внимание бе отделено на модула за управление на мултимедия:

- **Стриктна валидация:** Внедрени са проверки ~~както на клиентско ниво (JavaScript), така и на сървърно (C#),~~ които филтрират неподдържани файлови формати и ограничават размера на качените снимки.
- **Физическа оптимизация:** По време на тестването бе забелязан излишък от неизползвани файлове. Това доведе до създаването на механизъм за автоматично изтриване на снимки от сървъра при тяхното премахване от базата данни.

Адаптивност и UI/UX оптимизация

Интерфейсът премина през обстойна проверка за работа под различни разделителни способности:

- **Адаптивен дизайн:** Използвайки инструментите за разработчици (F12), беше тествано поведението на елементите (карти, таблици и бутони) при мобилни устройства и таблети, гарантирайки че Bootstrap мрежата се пренарежда правилно.
- **Визуална обратна връзка:** За по-добро потребителско изживяване бяха оптимизирани анимациите и преходите (напр. *grayscale* ефектът при маркиране на снимка за триене), за да бъде ясно на потребителя какво се случва във всеки момент.

3. Заключение

СЛАЙД 12: В настоящата дипломна работа беше разработено уеб приложението TravelMap — специализирана онлайн платформа за управление, организиране и споделяне на информация за туристически дестинации и персонализирани маршрути. Основната цел на проекта бе постигната чрез създаването на интуитивна и надеждна система, която улеснява потребителите в процеса на планиране и съхранение на техните пътешествия.

Чрез използването на съвременни технологии, беше изградена солидна архитектура, базирана на MVC шаблона. Това осигури ясно разделение между бизнес логиката, данните и интерфейса, гарантирайки лесна поддръжка и висока производителност. В хода на разработката бяха успешно реализирани комплексни механизми за работа с мултимедия, сигурност и оторизация на достъпа, както и динамично управление на лични и публични каталози чрез филтриране по държави, категории и тагове.

Планове за бъдещо развитие

Проектът полага солидна основа, която позволява надграждане в няколко ключови направления:

- **Интеграция с картографски услуги:** Внедряване на Google Maps API за визуализиране на дестинациите върху интерактивна карта и автоматично изчисляване на маршрути между тях.
- **Социални функционалности:** Добавяне на система за коментари, оценки и споделяне на каталози в социалните мрежи, което ще превърне платформата в активна общност.

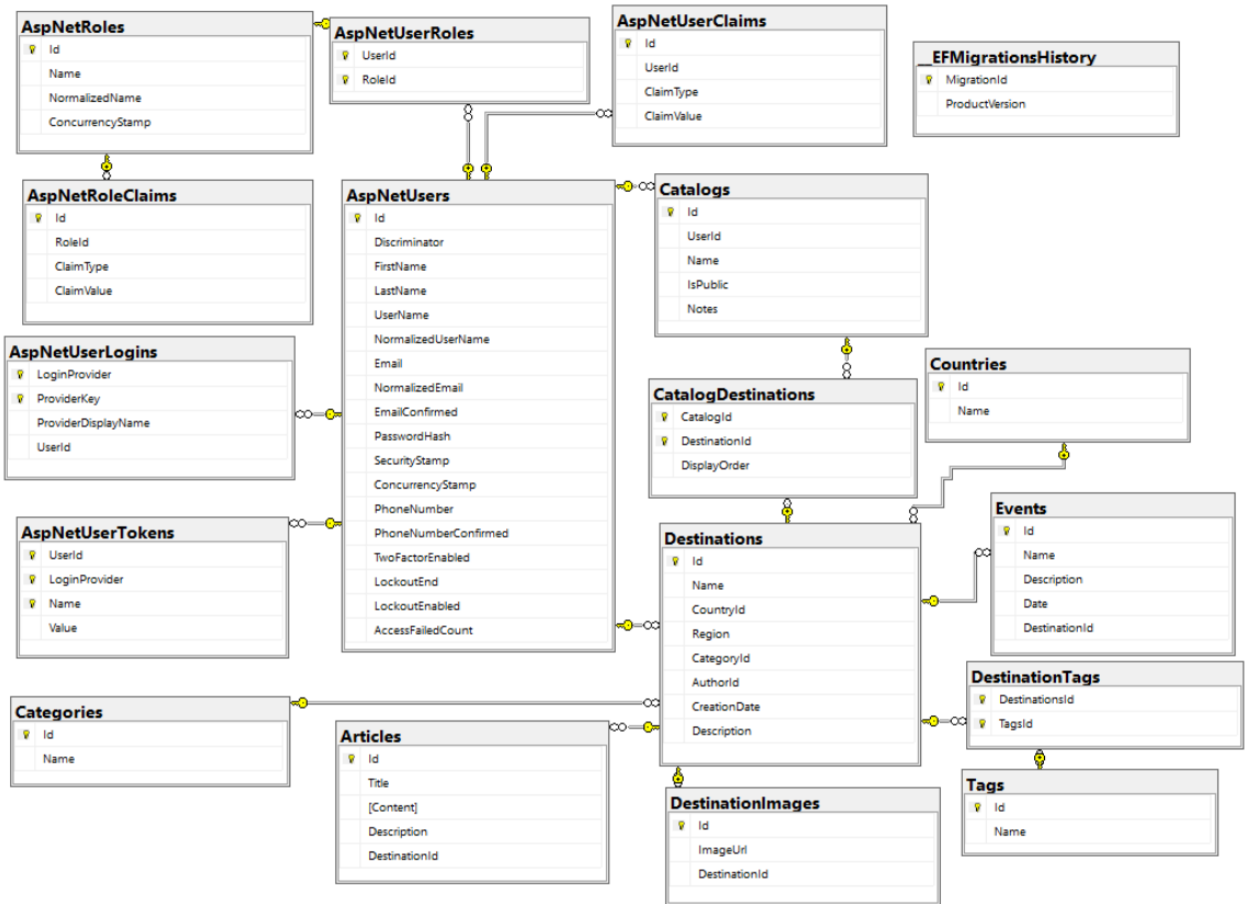
В заключение, TravelMap представлява завършен и мащабируем софтуерен продукт, който напълно отговаря на съвременните стандарти за уеб разработка и успешно адресира нуждите на съвременните пътешественици от организирано и естетично цифрово пространство за техните спомени.

4. Използвана литература

- Андерсън Рик – *Scaffold Identity in ASP.NET Core*: https://docs.microsoft.com/en-us/aspnet/core/security/authentication/scaffold-identity
- Андерсън Рик, Ларкин Кърк и Лароуз Даяна – *Session and state management in ASP.NET Core*: https://docs.microsoft.com/en-us/aspnet/core/fundamentals/app-state
- Наков Светлин – *Приложно програмиране (ASP.NET Core, EF Core)*: https://github.com/BG-IT-Edu/School-Programming/tree/main/Courses/Applied-Programmer
- Смит Стив – *Fundamentals of ASP.NET Core Middleware*: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/middleware/
- Bootstrap – *The most popular HTML, CSS, and JS library*: https://getbootstrap.com/
- Microsoft Docs – *File uploads in ASP.NET Core MVC*: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/file-uploads
- Microsoft Docs – *Interactive Image Cropping and Handling in .NET*: https://learn.microsoft.com/en-us/dotnet/architecture/blazor-for-web-forms-developers/media
- Microsoft Docs – *Model-View-Controller (MVC) Pattern*: https://learn.microsoft.com/en-us/aspnet/core/mvc/overview
- Microsoft Docs – *Validation in ASP.NET Core MVC*: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation
- Mozilla Developer Network (MDN) – *Using the FileReader API for client-side image preview*: https://developer.mozilla.org/en-US/docs/Web/API/FileReader
- Stack Overflow – *ASP.NET Core Multi-file upload techniques*: https://stackoverflow.com/questions/tagged/asp.net-core-mvc+file-upload
- W3Schools – *JavaScript DataTransfer API for handling file inputs*: https://www.w3schools.com/jsref/obj_datatransfer.asp

5. Приложения

Приложение 1: Схема на базата данни



Приложение 2: Архитектура на приложението

Приложение 3: Установяване на зависимости между таблиците

```
// blocks deletion of category if there exist Destinations with it
builder.Entity<Destination>()
    .HasOne(d => d.Category)
    .WithMany(c => c.Destinations)
    .HasForeignKey(d => d.CategoryId)
    .OnDelete(DeleteBehavior.Restrict);

// set destionation's author to null if user is deleted
builder.Entity<Destination>()
    .HasOne(d => d.Author)
    .WithMany(a => a.Destinations)
    .HasForeignKey(d => d.AuthorId)
    .OnDelete(DeleteBehavior.SetNull);

// if destination is deleted - image records are deleted too
builder.Entity<DestinationImage>()
    .HasOne(di => di.Destination)
    .WithMany(d => d.Images)
    .HasForeignKey(di => di.DestinationId)
    .OnDelete(DeleteBehavior.Cascade);

// if user is deleted - his catalogs are deleted too
builder.Entity<Catalog>()
    .HasOne(c => c.ApplicationUser)
    .WithMany(au => au.Catalogs)
    .HasForeignKey(c => c.UserId)
    .OnDelete(DeleteBehavior.Cascade);

// unique pair (catalog id, destination id)
builder.Entity<CatalogDestination>()
    .HasKey(cd => new { cd.CatalogId, cd.DestinationId });
```

Приложение 4: Сортиране и оптимизиране на заявките при дестинациите

```
ViewData["CurrentSort"] = sortOrder;
ViewData["NameSortParm"] = String.IsNullOrEmpty(sortOrder) ? "name_desc" : "";
ViewData["DateSortParm"] = sortOrder == "Date" ? "date_desc" : "Date";
ViewData["CountrySortParm"] = sortOrder == "Country" ? "country_desc" : "Country";
ViewData["PopSortParm"] = sortOrder == "Popularity" ? "pop_desc" : "Popularity";

var anonQuery = _context.Destinations
    .Select(d => new
    {
        Destination = d,
        PopCount = d.CatalogDestinations.Count(),
        ImgCount = d.Images.Count()
    });

anonQuery = sortOrder switch
{
    "name_desc" => anonQuery.OrderByDescending(a => a.Destination.Name),
    "Date" => anonQuery.OrderBy(a => a.Destination.CreationDate),
    "date_desc" => anonQuery.OrderByDescending(a => a.Destination.CreationDate),
    "Country" => anonQuery.OrderBy(a => a.Destination.Country.Name),
    "country_desc" => anonQuery.OrderByDescending(a => a.Destination.Country.Name),
    "Popularity" => anonQuery.OrderBy(a => a.PopCount),
    "pop_desc" => anonQuery.OrderByDescending(a => a.PopCount),
    _ => anonQuery.OrderByDescending(a => a.PopCount)
};

var results = await anonQuery
    .AsNoTracking()
    .Select(a => a.Destination)
    .Include(d => d.Category)
    .Include(d => d.Country)
    .Include(d => d.Images)
    .Include(d => d.Tags)
    .Include(d => d.Author)
    .Include(d => d.CatalogDestinations)
    .AsSplitQuery()
    .ToListAsync();

return View(results);
```

Приложение 5: Групиране на характеристиките на дестинация в общ обект

```
public async Task<IActionResult> Index()
{
    var model = new DestinationSpecifications
    {
        Countries = await _context.Countries
            .OrderBy(b => b.Name)
            .ToListAsync(),

        Categories = await _context.Categories
            .OrderBy(ct => ct.Name)
            .ToListAsync(),

        Tags = await _context.Tags
            .OrderBy(m => m.Name)
            .ToListAsync()
    };

    return View(model);
}
```

Приложение 6: Филтриране в реално време

```
<script>
    const searchInput = document.getElementById('searchInput');
    const cards = document.querySelectorAll('.destination-card');
    const noResults = document.getElementById('noResults');
    const tagBtns = document.querySelectorAll('.tag-filter-btn');

    searchInput.addEventListener('input', (e) => {
        filterContent(e.target.value.toLowerCase());
        updateActiveTag('none');
    });

    function filterByTag(tagName, btn) {
        searchInput.value = tagName;
        filterContent(tagName);
        updateActiveTag(tagName);
    }

    function filterContent(term) {
        let count = 0;
        cards.forEach(card => {
            const data = card.getAttribute('data-search');
            const match = data.includes(term);
            card.classList.toggle('d-none', !match);
            if (match) count++;
        });
        noResults.classList.toggle('d-none', count > 0);
    }

    function updateActiveTag(tagName) {
        tagBtns.forEach(btn => {
            btn.classList.toggle('active', btn.getAttribute('data-tag') === tagName);
        });
    }

    function clearFilters() {
        searchInput.value = '';
        filterContent('');
        updateActiveTag('all');
    }
</script>
```

Ръководство за потребителя

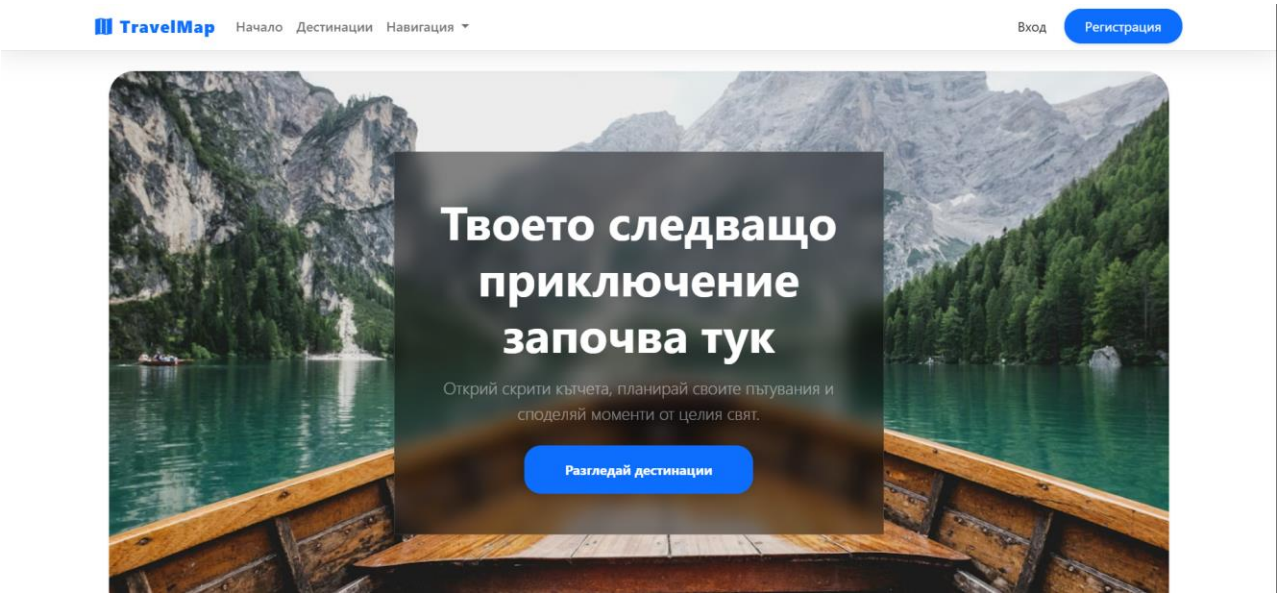
Администраторски акаунт:

- Имейл: admin@travel.com
- Парола: Admin123!

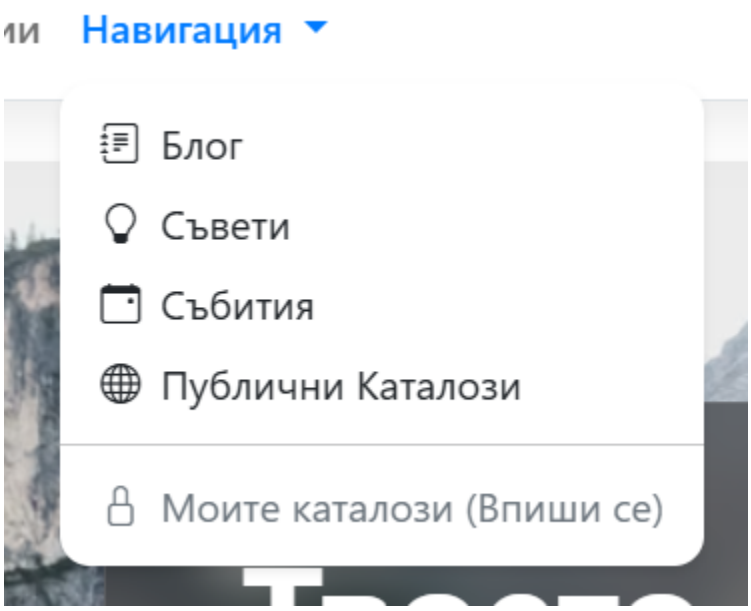
Акаунт без привилегии:

- Имейл: user@travel.com
- Парола: User123!

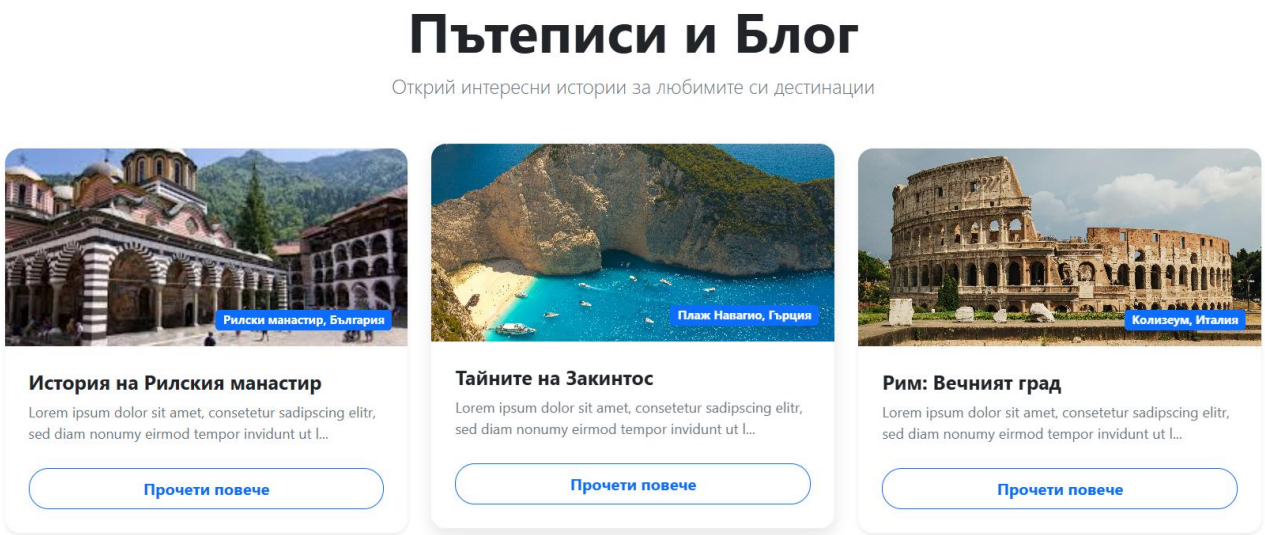
Когато потребител отвори уебсайта той е пренасочен до началната страница, която има връзки към списъка с всички дестинации и падащ списък с навигация, който се активира с натискане върху него.



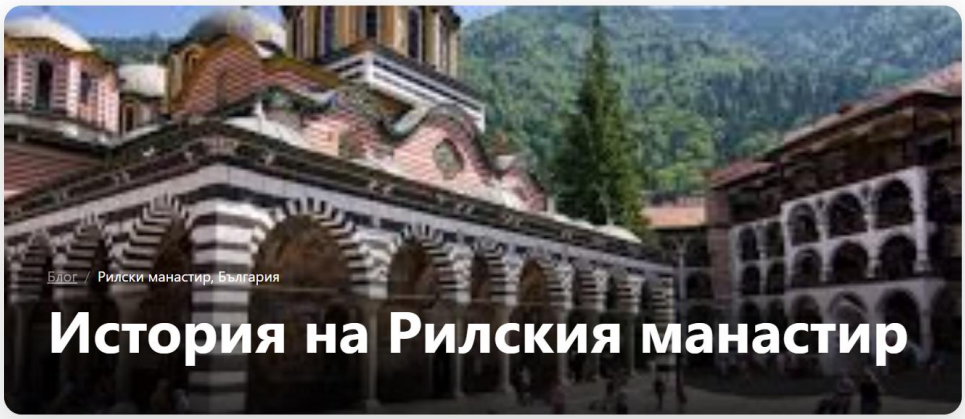
Той съдържа връзки към блога със статиите за дестинации, списък със съвети за туристи и форма за контакт, всички събития, които ще се случат в дестинациите и списък с публични каталози (маршрути) на потребителите.



Страницата с блога показва статии за дестинациите, както и първата тяхна снимка.

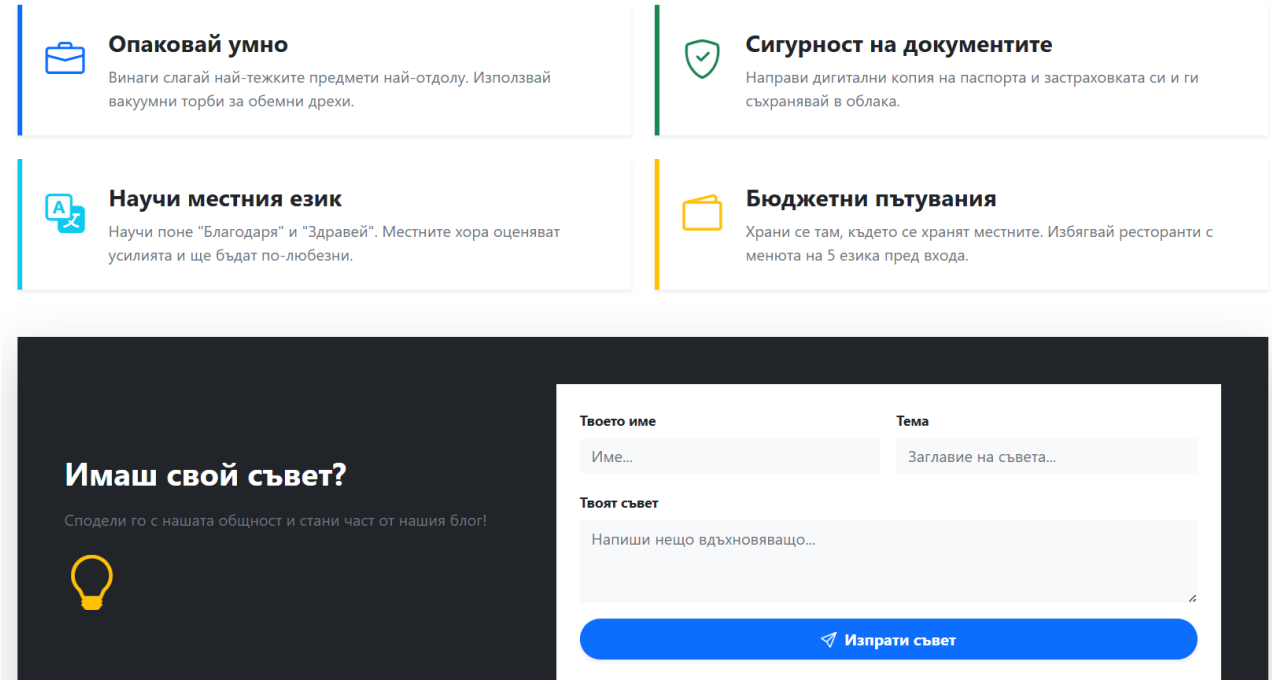


С натискането на „Прочети повече“ потребителят отива до страницата с пълната статия за този обект.

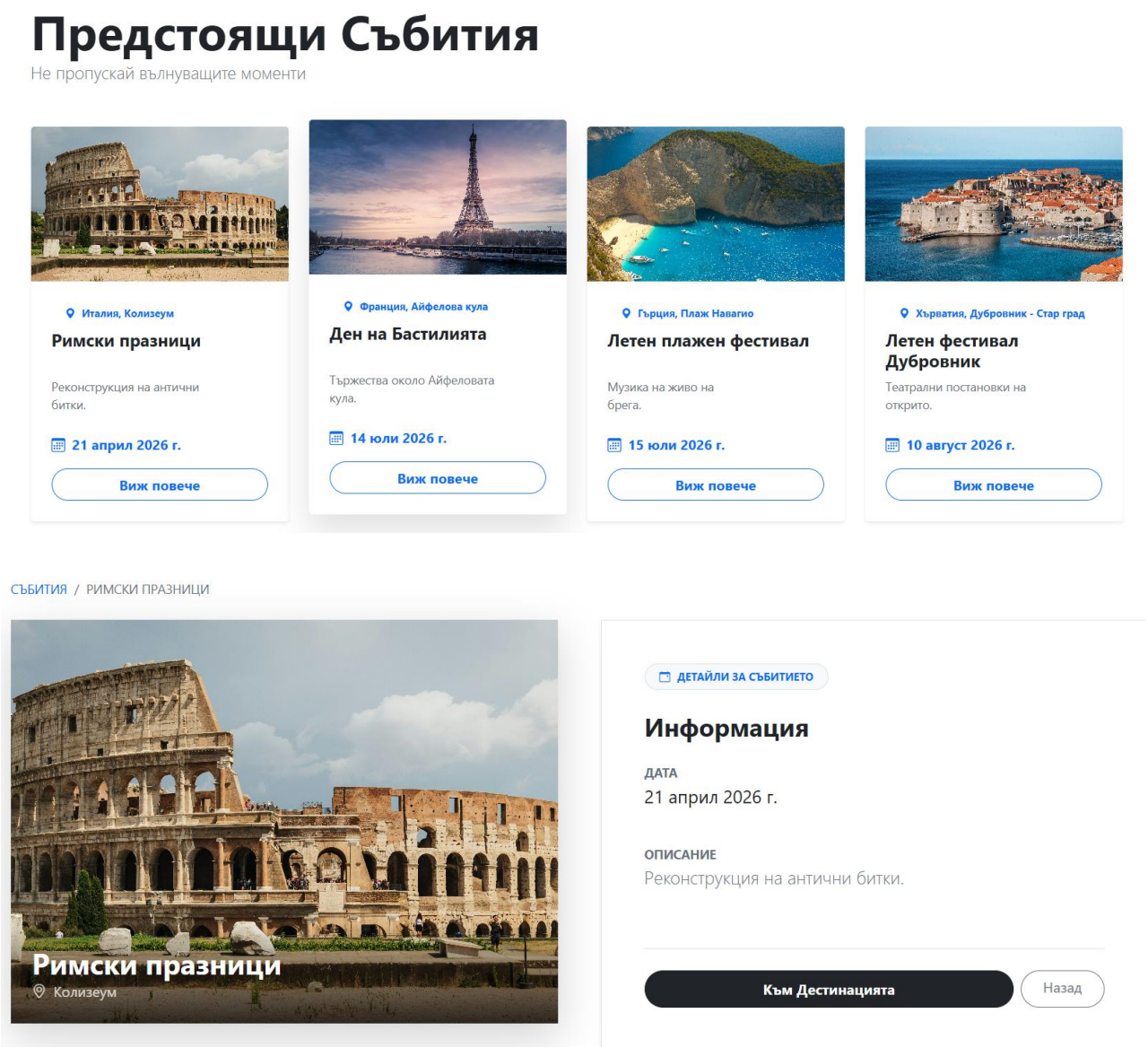


"Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam voluptua. At vero

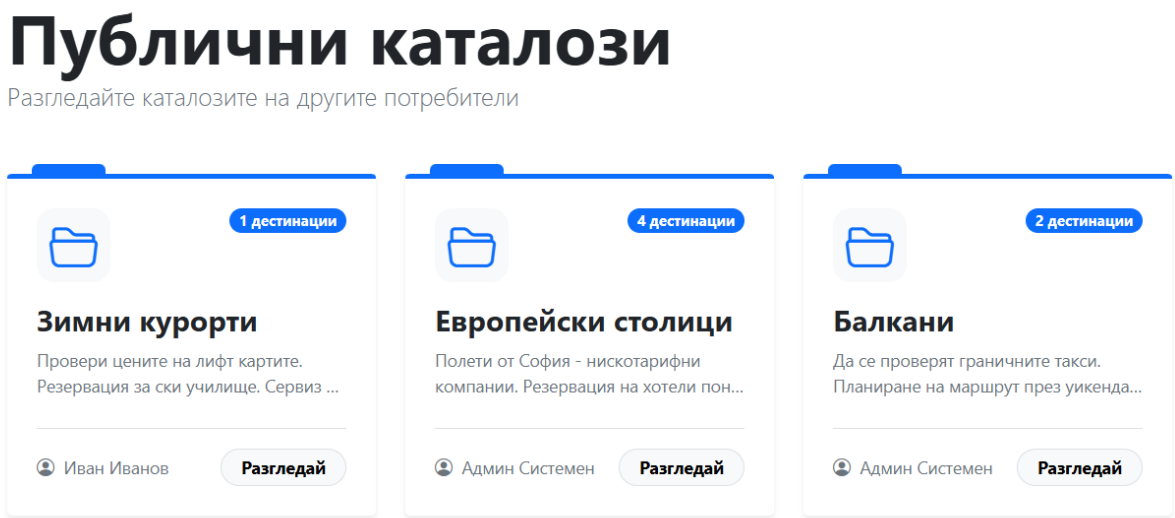
Страницата за съветите е кратък списък с насоки за потребителите и форма за контакт с администраторите за добавяне на нов съвет.



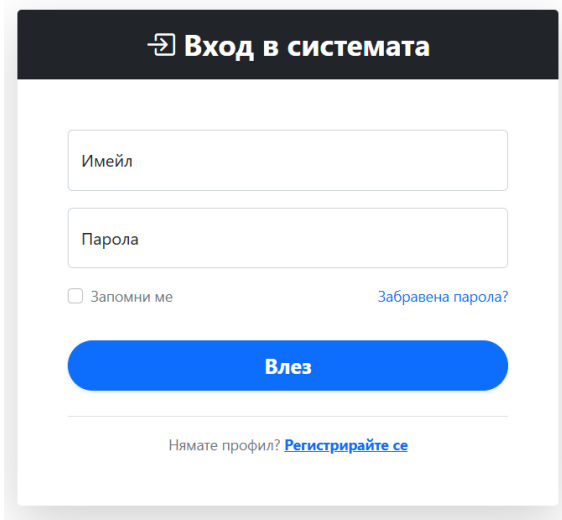
Страницата със събитията показва къде и кога ще има тържество с допълнителни детайли след натискане на бутона „Виж повече“.

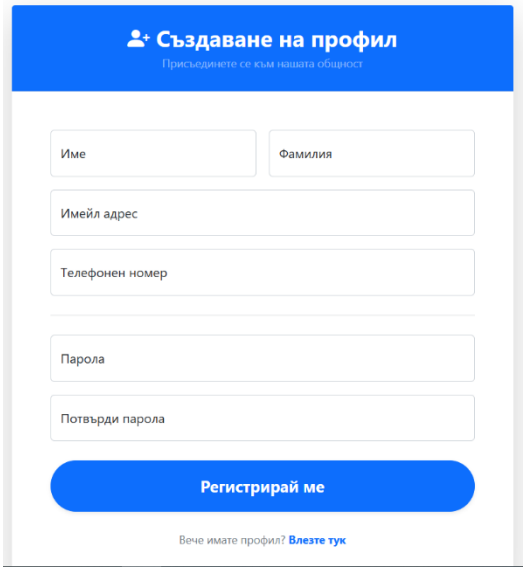


Последно меню от навигацията за гост представлява списък с всички публични потребителски маршрути.

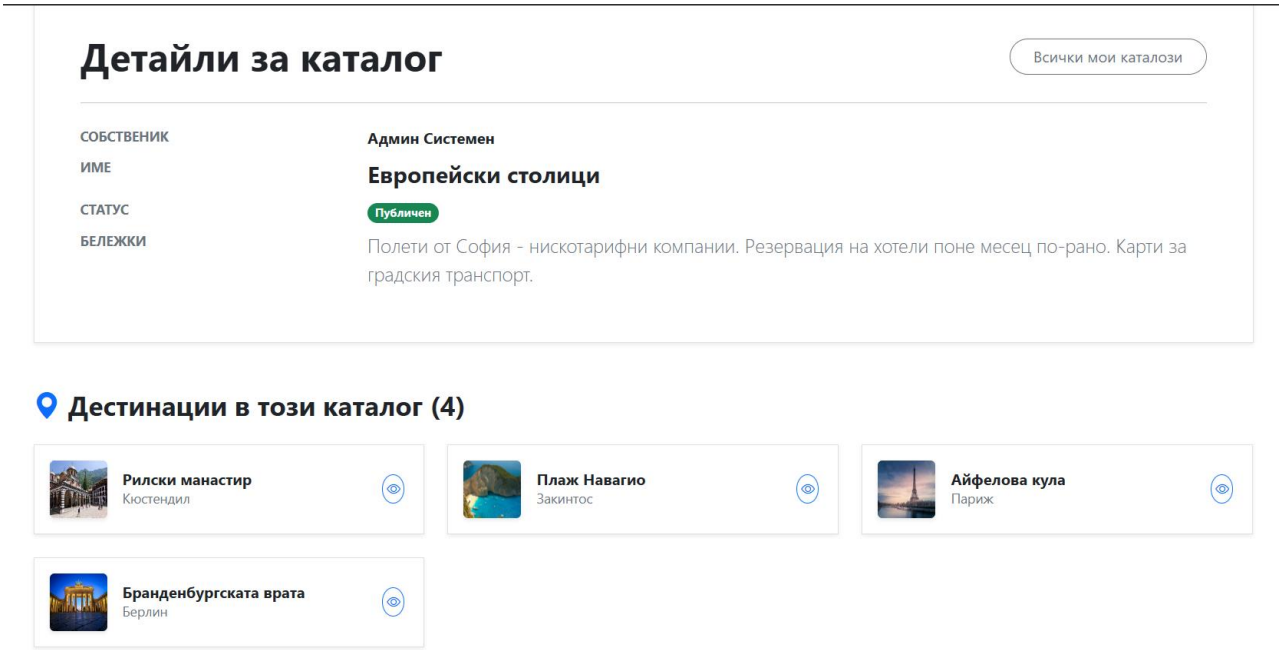


Когато обаче потребителя се опита да отвори даден каталог той бива изпратен до формата за вписване и ако няма акаунт трябва да избери тази за регистриране.

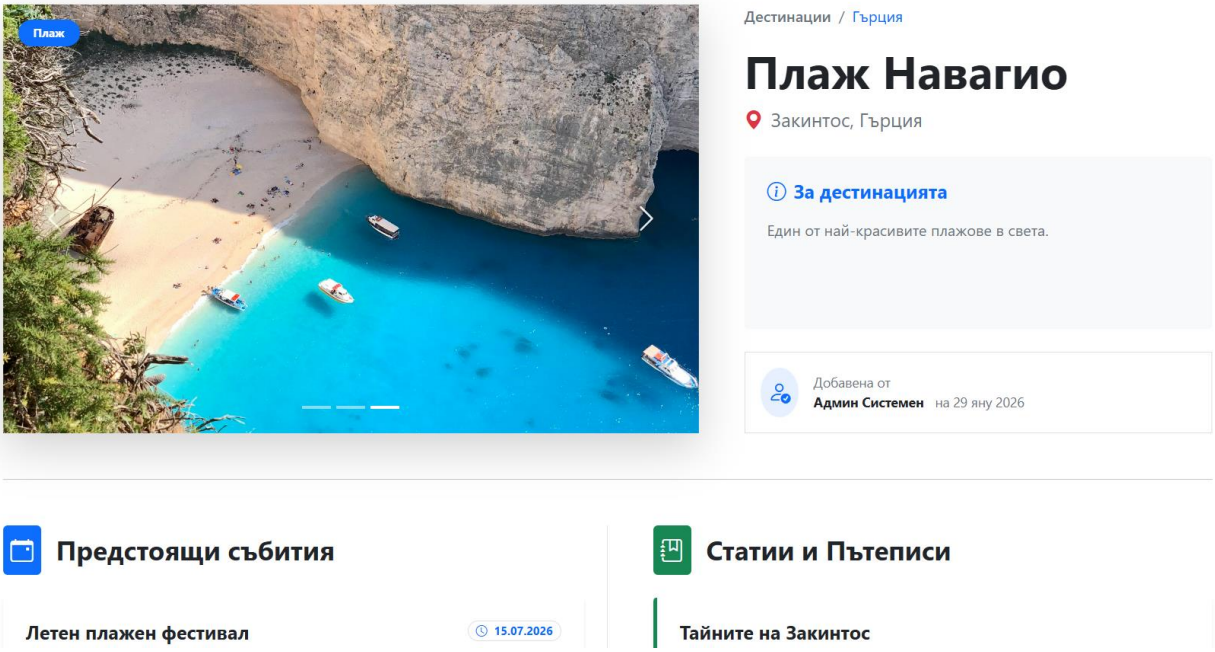




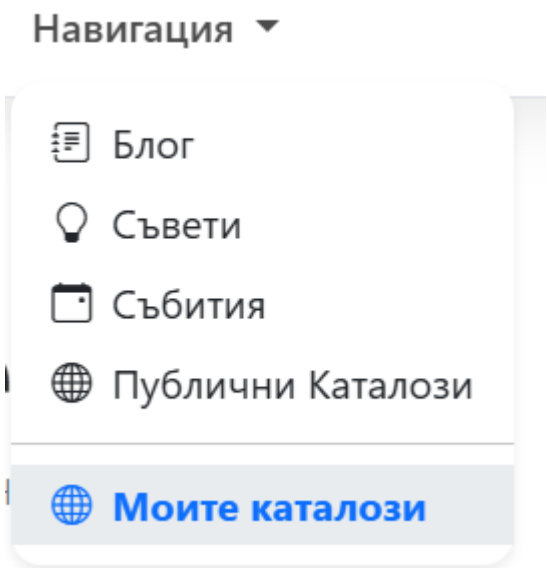
Оттам вече потребителят може да разглежда маршрутите, бележките и дестинациите в тях.

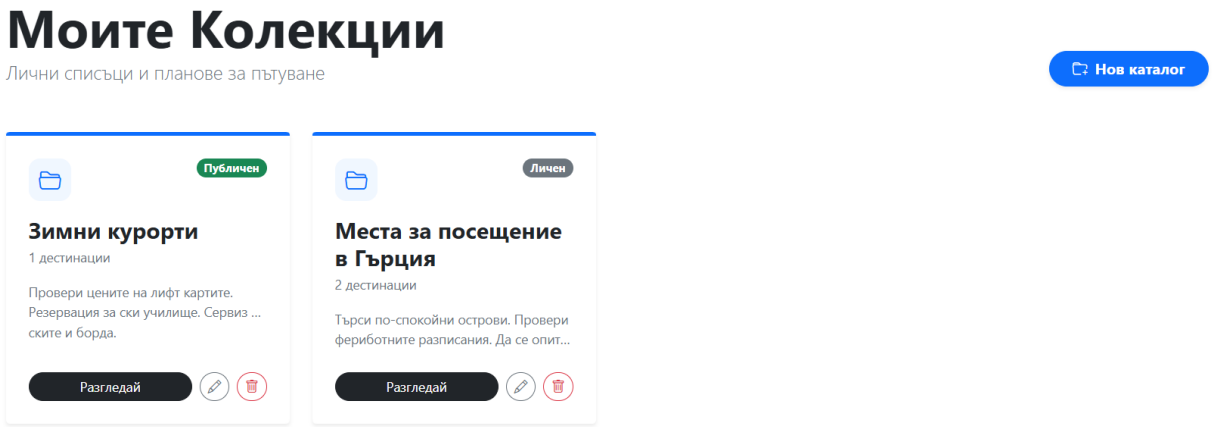


С натискането върху окото до дестинацията, клиентът е пренасочен до страницата с повече информация за това място, където може да прочете цялата информация за него, заедно с всички блогове и събития. Които ще се проведат там.

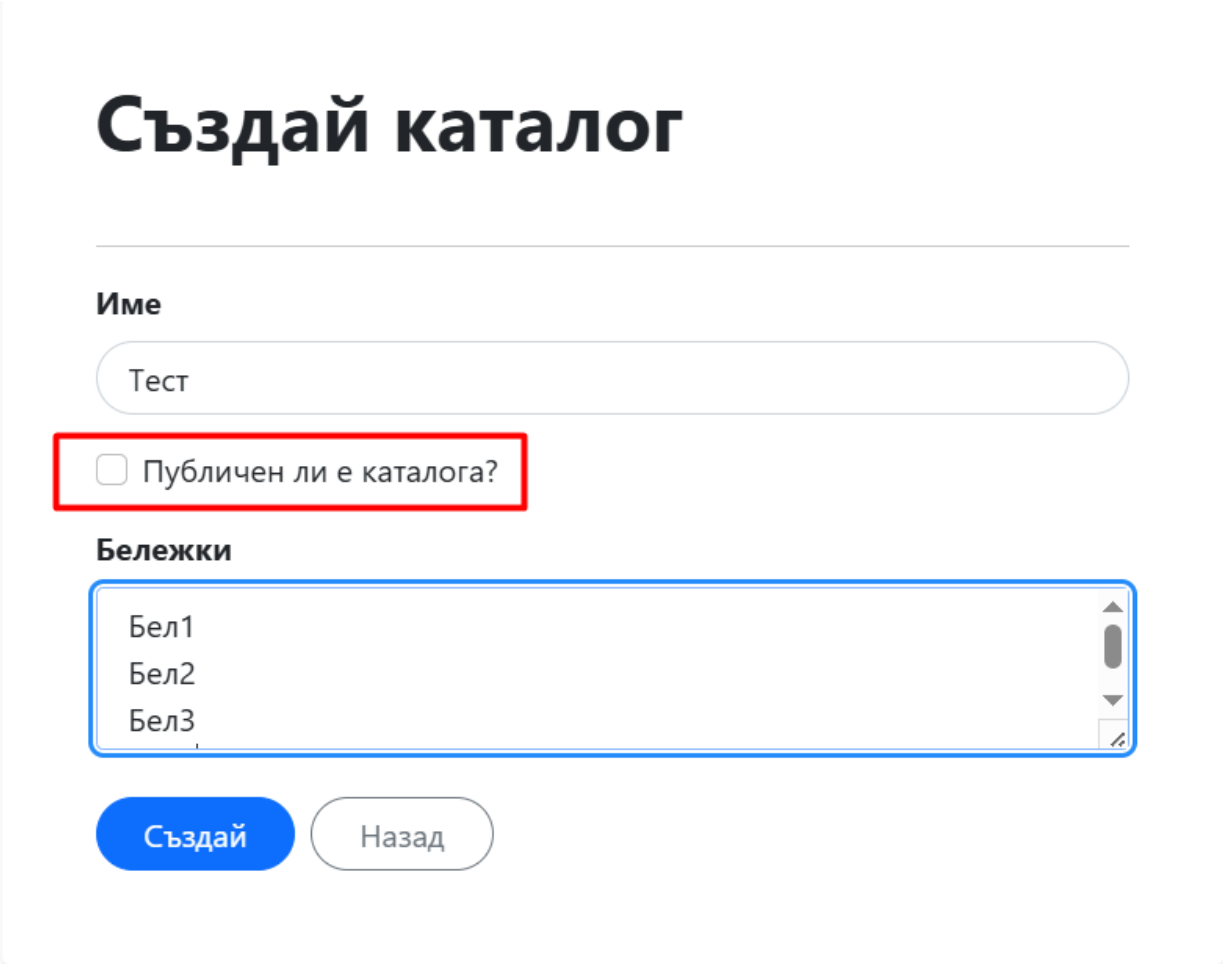


Когато иска да създаде свой собствен каталог, той трябва да натисне върху списъка за навигация отново и да натисне в новопоявилата се връзка „Моите каталози“.

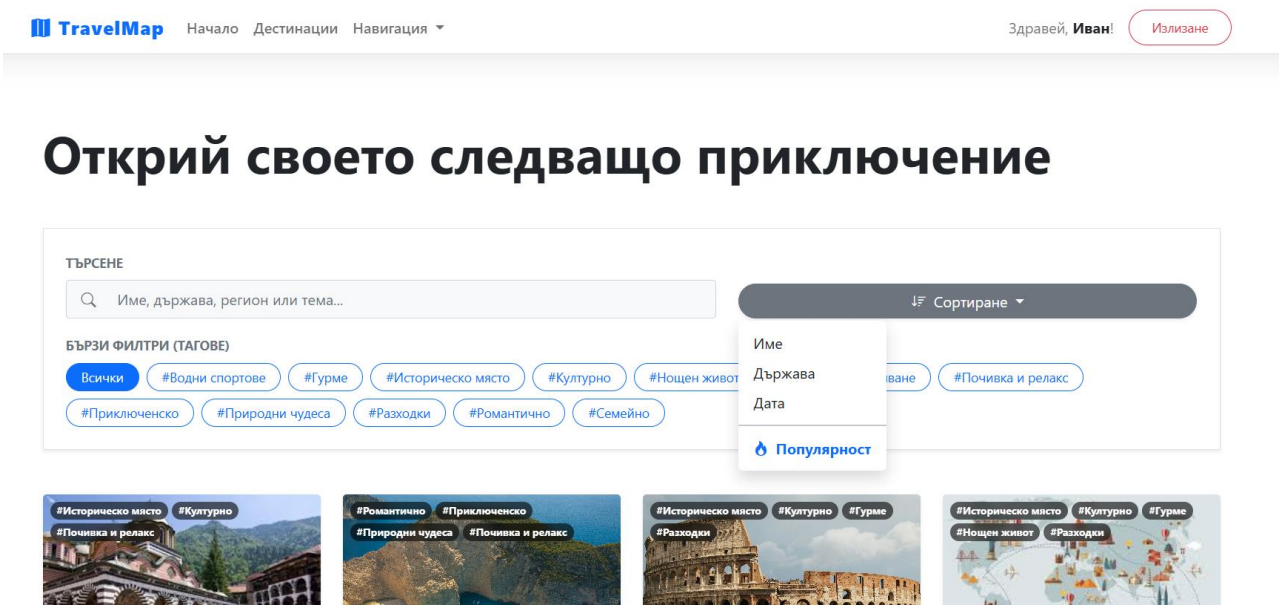




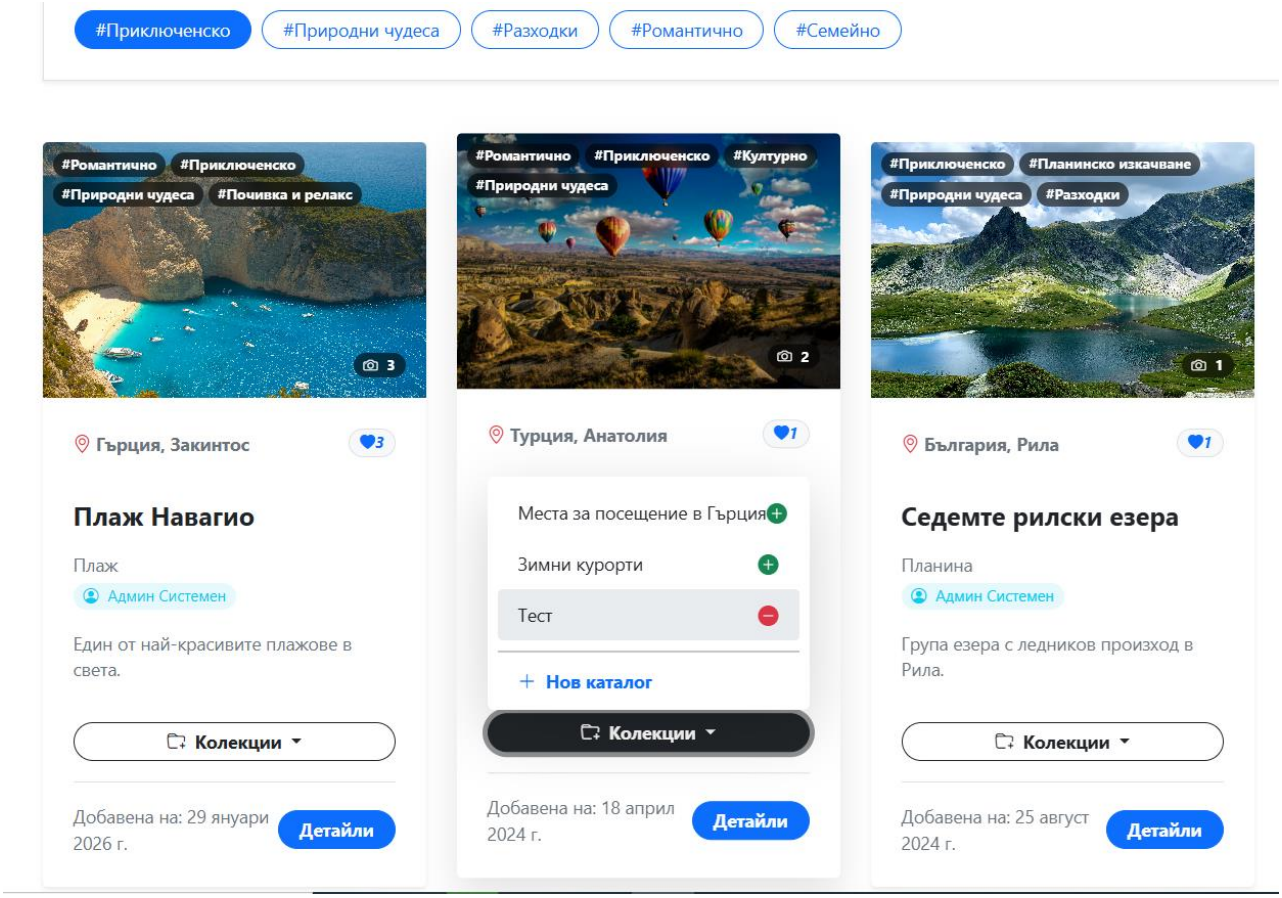
Оттам се избира „Нов каталог“ и попълва данните за своя каталог като избира и видимостта му.



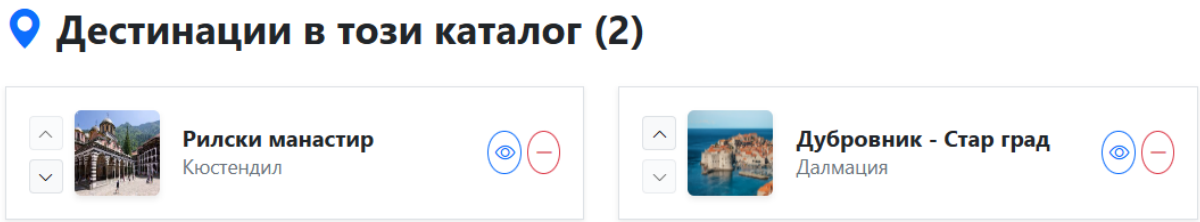
След това потребителя отива в дестинации с натискането на бутона в горната лента, където има възможността за подрежда или групира обектите по определени критерии.



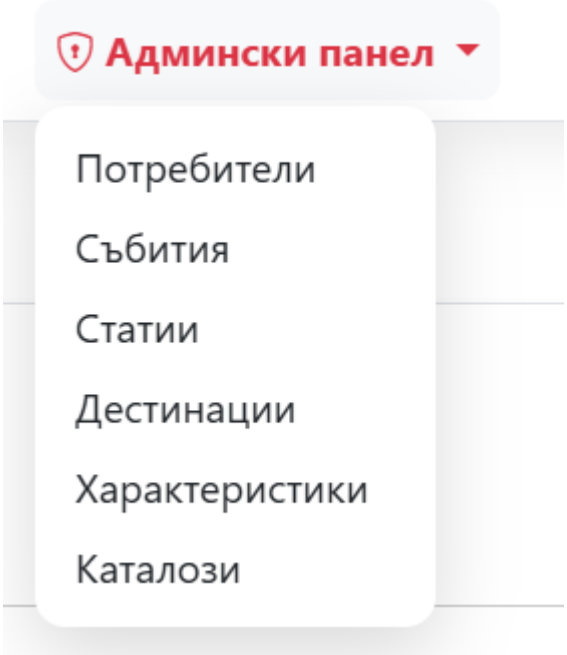
След като намери желаната дестинация натиска върху бутона „Колекции“ и избира в кои маршрути да я добави или премахне.



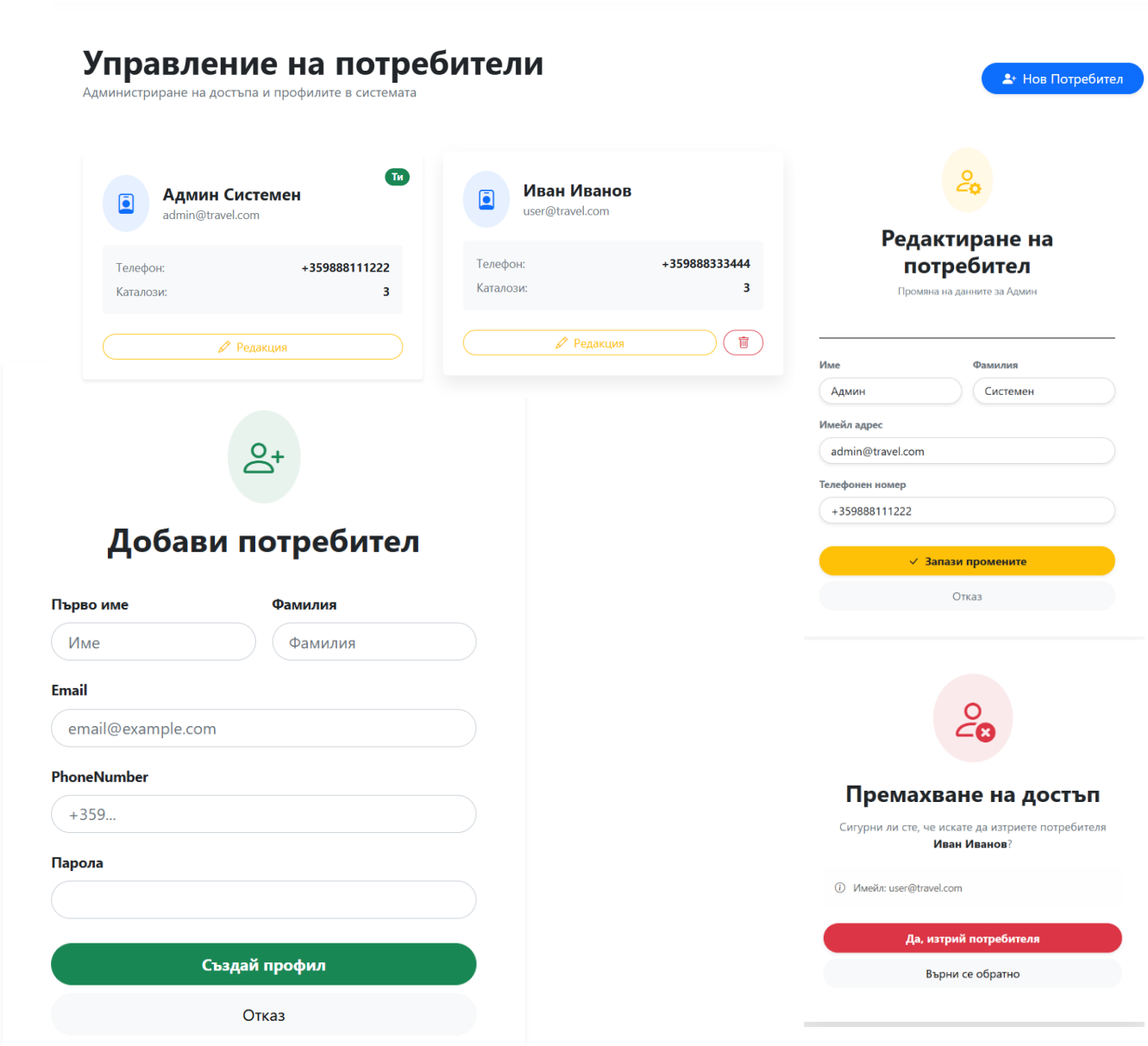
Когато потребителя е готов и отвори своя маршрут, той има възможността да пренарежда дестинациите в него с натискане върху стрелките.



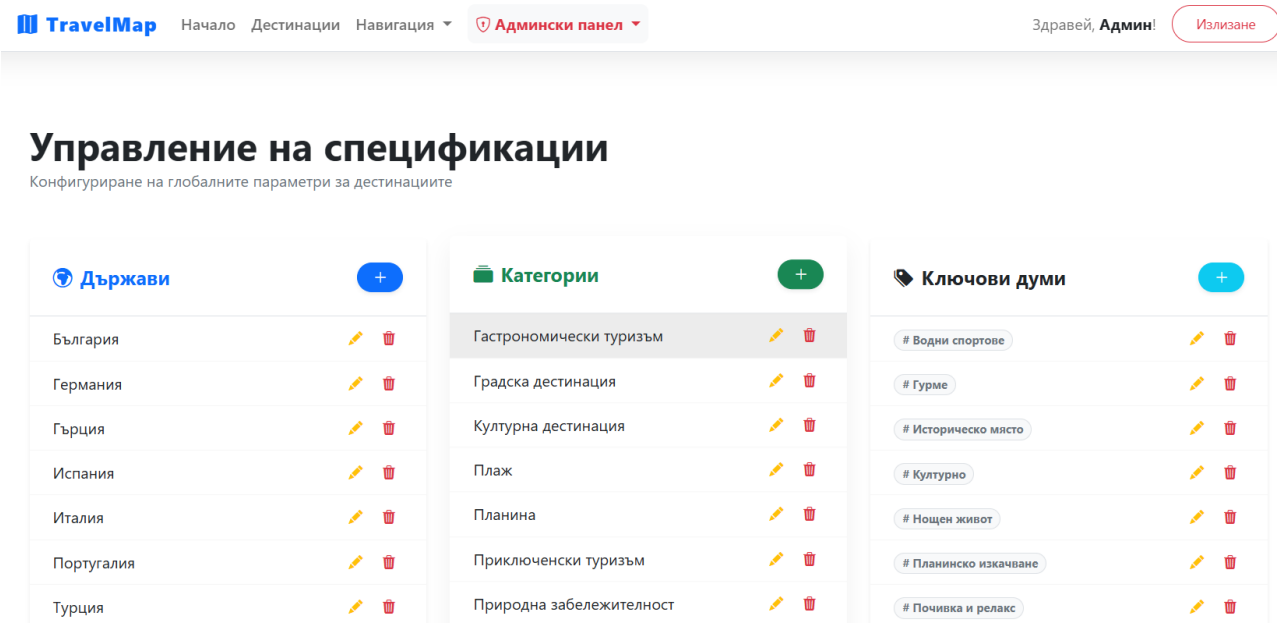
С това функционалностите на обикновения потребител приключват. Когато администраторски акаунт посети сайта, от началото той има достъп до още един падащ списък – с администския панел.



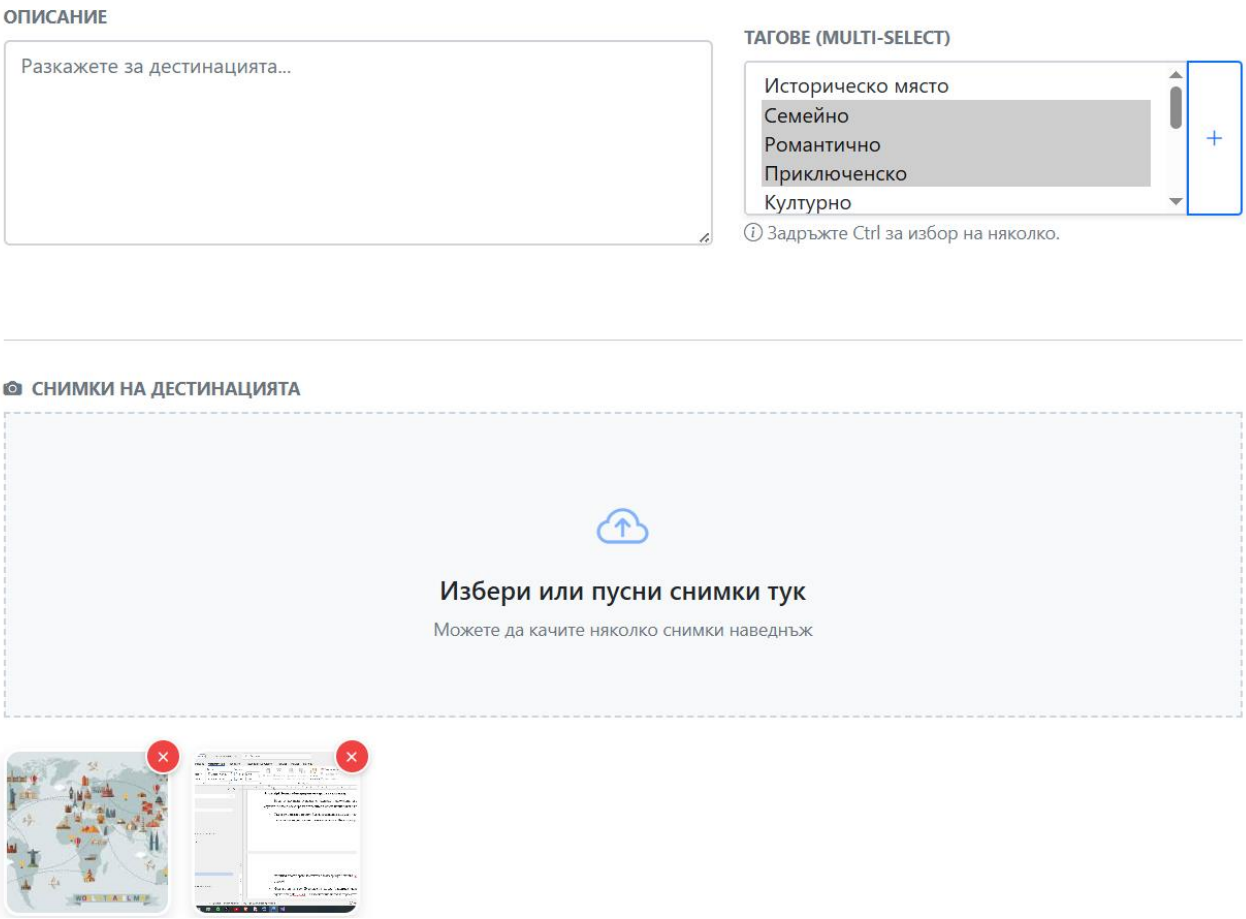
Оттам той може да управлява всички записи – да ги прочита, променя, изтрива или създава нови. Пример за това е „Потребители“



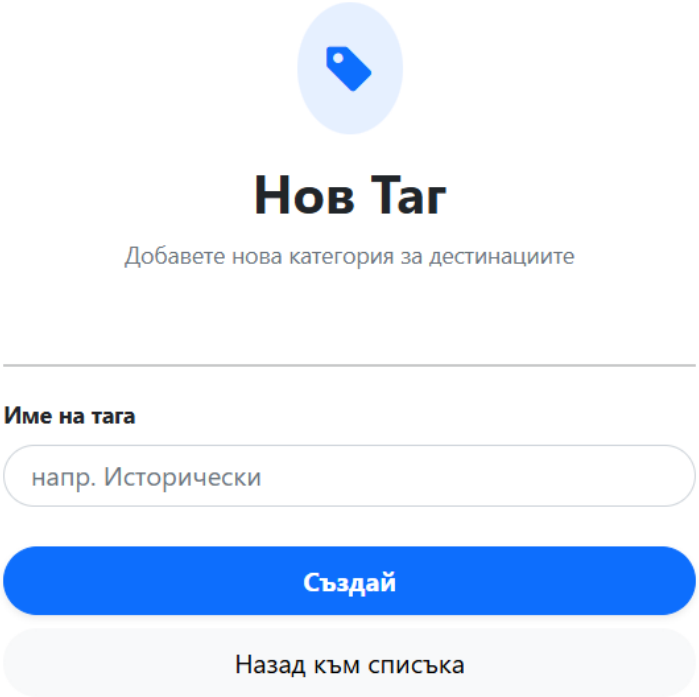
Аналогично същото се показва за статии и събития. От страницата „Характеристики“ може да оперира върху предварително записаните стойности, като държави или тагове.



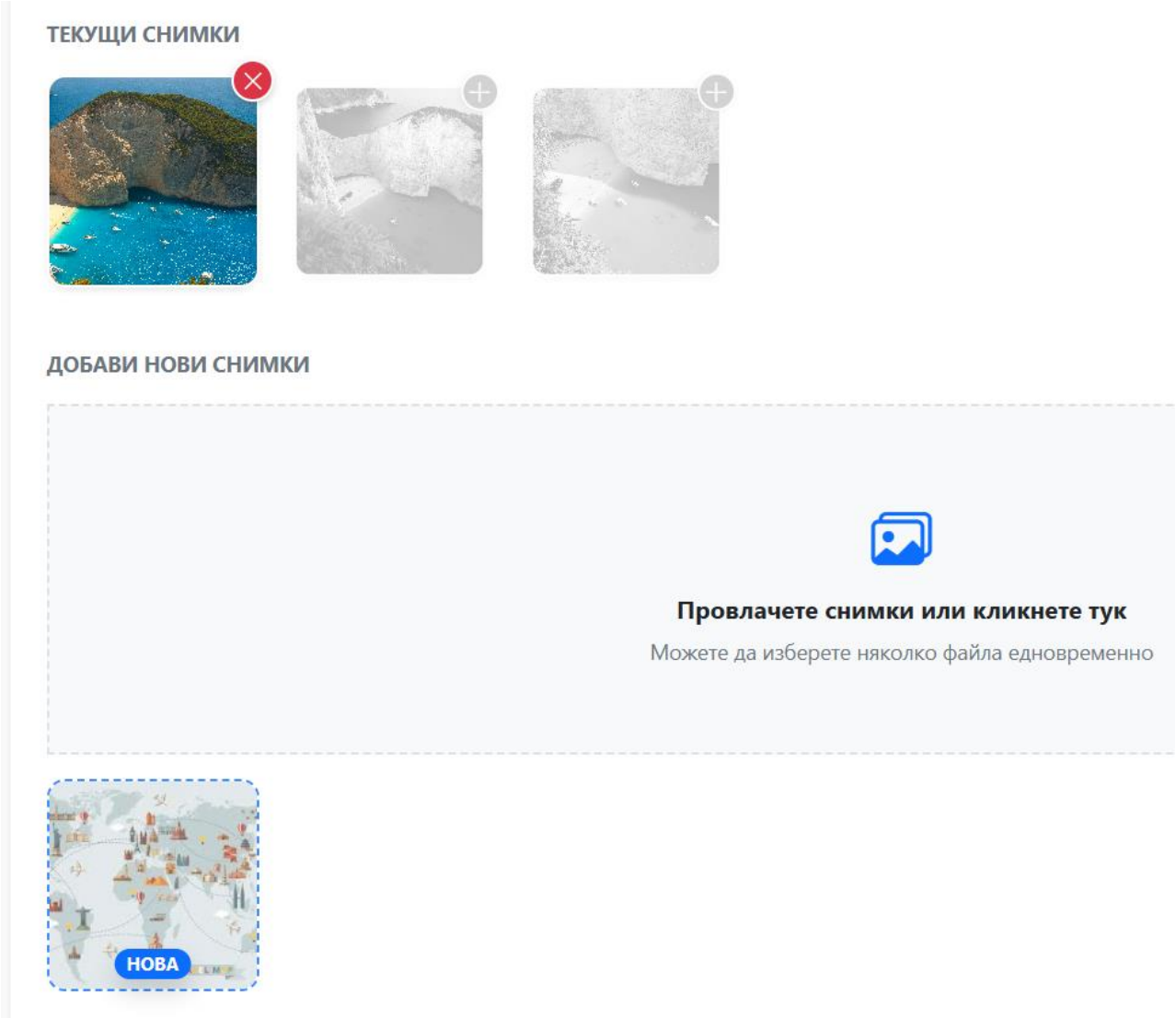
Панела за дестинациите показва всички забележителности с възможността за създаване на нови с добавяне на много снимки



и в случай че няма характеристика, която да отговаря за мястото, с натискане на бутона „+“ до падащия списък със стойностите ѝ, потребителят е отведен до форма за създаване на нова такава.



Другото нещо, което администратора може да прави е да променя данни за съществуваща локация, като замяна на снимка.



В случай на грешка, като премахване на грешното изображение, администратора може лесно да го върне с повторно натискане върху него. В реално време се визуализира какви ще са новите изображения за местоположението.

Последната част от панела е менюто за управление на маршрути, където потребителя може да прави същото върху всеки каталог, което може да прави върху своите каталози.

TravelMap Начало Дестинации Навигация Админски панел Здравей, **Админ**! Излизане

Управление на каталози

+ Нов Каталог

Име	Собственик	Статус	Бележки	Действия
Балкани	admin@travel.com	Публичен	Да се проверят граничните такси. Планиране на маршрут през уикенда. Включи местни ресторанти.	
Историческо наследство	admin@travel.com	Личен	Списъкът включва обекти под защитата на ЮНЕСКО. Провери работното време на музеите. Носи удобни обувки.	
Европейски столици	admin@travel.com	Публичен	Полети от София - нискотарифни компании. Резервация на хотели поне месец по-рано. Карти за градския транспорт.	
Места за посещение в Гърция	user@travel.com	Личен	Търси по-спокойни острови. Провери фериботните разписания. Да се опита местния октопод.	
Зимни курорти	user@travel.com	Публичен	Провери цените на лифт картите. Резервация за ски училище. Сервиз на ските и борда.	
Тест	user@travel.com	Личен	Бел1 Бел2 Бел3 Бел4	